

SCHOOL OF COMPUTER SCIENCE

Epicycles

A Bidirectional DSL for Geometric Art

Alexander Wood

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Engineering in the Faculty of Engineering **worth 40CP**.

Wednesday 7th October, 2026

Abstract

We present a bidirectional embedded Domain Specific Language (eDSL) for creating art by combining epicycles (i.e. Fourier series), allowing programmatic generation and direct manipulation. We use epicycles to ensure unambiguous edits, addressing limitations within other bidirectional art systems such as Sketch-n-Sketch. We implement two bidirectional programming semantics for our DSL, a heuristic-based approach and a more recent delta-based approach, and compare their effectiveness at resolving ambiguity and providing a predictable user experience. We also formally verify our implementation using the Lean theorem prover, proving core properties to ensure predictable behaviour. Finally, we evaluate the effectiveness of the semantics and language design, under the lenses of bidirectional programming and creative coding.

Dedication and Acknowledgements

This work is dedicated to my supervisor Samantha Frohlich for her helpful support and advice, and to my partner Moon for their unwavering support and encouragement throughout the year.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Alexander Wood, Wednesday 7th October, 2026

AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.

Alexander Wood, Wednesday 7th October, 2026

Contents

1 Introduction **1**

2 Background **3**
2.1 Creative Coding and Direct Manipulation . . . 3
2.2 The View-Update Problem . . . 3
2.3 Bidirectional Programming and Optics . . . 4
2.4 Epicyclic Geometry and the Fourier Transform . . . 5
2.5 Embedded DSLs in Haskell . . . 7

3 Project Execution **9**
3.1 Architectural Evolution and Scope . . . 9
3.2 Domain Restriction: Motivation . . . 11
3.3 UI and Interaction Model . . . 12
3.4 DSL Design . . . 13
3.5 Semantics . . . 19
3.6 Non-Deterministic Semantics . . . 21
3.7 Delta-Based Semantics . . . 22
3.8 Comparison of Strategies . . . 25
3.9 Compilation and Evaluation Pipeline . . . 27

4 Critical Evaluation **29**
4.1 Formal Properties of the Backwards Semantics . . . 29
4.2 Correctness of the non-deterministic semantics . . . 29
4.3 Correctness of the Delta Semantics . . . 31
4.4 Expressivity . . . 32
4.5 Direct Manipulation and User Experience . . . 36
4.6 Metrics . . . 37
4.7 Automated Testing . . . 39
4.8 Summary . . . 39

5 Conclusion **40**

A Appendix A: AI Prompts/Tools **42**

List of Figures

3.1 The main UI of our system, showing the code pane on the left, the rendered curve in the middle, and extra controls on the right. . . . 12
3.2 The UI clearly shows the connection between the code and the visual output, highlighting the relevant code and construction element when hovering. In this example, we are hovering over the phase parameter (the arm) of the second epicycle. . . . 13
3.3 The output of an example curve defined in our DSL. . . . 18
3.4 Delta language for our DSL . . . 23
3.5 Delta semantics for the numeric part of the language, adapted from [19] . . . 23
3.6 Backwards evaluation rules for the numeric part of our delta language, adapted from [19]. 26
3.7 Backwards evaluation rules for the curve part of our delta language, adapted from [19]. For brevity, we abbreviate *epicycle* as *epi* . . . 27
3.8 Compilation and evaluation pipeline. The upper path is the one-shot compilation from HOAS to a numbered AST. The lower loop runs every frame: `evalScene` evaluates the AST into a visual scene, which is rendered to the `Canvas`; user interactions produce `SceneEdit`s which are propagated back through the AST via the chosen backward strategy, producing an updated `CurveState`. . . . 28

4.1 A simplified version of our Poster Day poster, generated with the DFT-based functionality of our DSL. . . . 33
4.2 The text ”Hello” interpolated with the Discrete Fourier Transform with 5 cycles per glyph (left) and 10 cycles per glyph (right). . . . 33
4.3 A straight line at angle $\pi/2$ formed of two epicycles, with construction enabled . . . 34
4.4 A deltoid curve formed of two epicycles, with construction enabled . . . 34
4.5 A curve with a shared parameter used multiple times. The decreasing radii create a harmonic decay effect. . . . 35
4.6 A snowflake-like curve generated by exponentially decaying a shared radius parameter across 4 epicycles. . . . 35
4.7 An asymmetric curve generated by sharing a parameter across the phase variable. . . . 35
4.8 A Guilloché curve generated by sharing a radius parameter across 5 epicycles . . . 36
4.9 Editing the frequency parameter ω_2 of an epicycle produces a variety of different curves. . 36

List of Tables

4.1 AST Size Growth under FuseDM semantics. The zero standard deviation across runs confirms convergence to a structural fixed point. 38
4.2 Intent leakage and structural reversibility when oscillating on shared scenes. 38
4.3 PutPut violation rates across 984 tests. 39

Ethics Statement

This project did not require ethical review, as determined by my supervisor, Samantha Frohlich.

Supporting Technologies

- I used the **Lean4** theorem prover to formally model the semantics of the DSL and mechanically verify core properties.
- The interactive graphical user interface was built using the **Dear ImGui** library (via the `dear-imgui` Haskell bindings), operating on top of **SDL2** and **OpenGL3**.
- I used a modified fork of the reference implementation of the *'Embedding by Unembedding'* paper[11]: `https://github.com/kztk-m/EbU` to implement the Higher-Order Abstract Syntax (HOAS) frontend into the First-Order Abstract Syntax (FOAS) backend of the DSL.
- The code-pane visualisation of the AST was generated using the *prettyprinter* library, which allowed for dynamic, syntax-highlighted (manually implemented) rendering of the source code.
- I used *hedgehog* to perform property-based testing alongside the *sydtest* framework. This was used to empirically validate the behaviour of the implementation, supporting the formal proofs in Lean.
- I used the **Nix** package manager to define a reproducible, declarative build environment, and provide useful development utilities.

Notation and Acronyms

AST	:	Abstract Syntax Tree
BNF	:	Backus-Naur Form
DFT	:	Discrete Fourier Transform
DSL	:	Domain-Specific Language
FOAS	:	First-Order Abstract Syntax
HOAS	:	Higher-Order Abstract Syntax
SnS	:	Sketch-n-Sketch, a bidirectional creative coding environment
FuseDM	:	FuseDM, a bidirectional update strategy based on delta updates

Chapter 1

Introduction

The field of Creative Coding – creating art, music, or other creative media using code – often faces an inconvenience that hinders the creative process: lack of support for *Direct Manipulation*, i.e. being able to edit the graphical or auditory output of programs as-is via direct interaction. A motivating example is the programmatic music editor TidalCycles:[1] while the live coding environment facilitates rapid iteration, the only way to alter the output is by textually editing the code - there is no support for, for example, clicking and dragging to modulate the pitch of a tone.
Recent developments in the programming languages research field of *bidirectional transformations* have motivated attempts to fix this. A prominent example is Sketch-n-Sketch[8], which provides a bidirectional interface to creative coding, where users can write code in a custom Elm-like DSL, composing shapes on a canvas, or drag on the visual canvas to add new shapes or modify existing ones. A change in one pane synchronises across both outputs, theoretically allowing the artist to use the most effective interaction form without compromise.
However, systems like Sketch-n-Sketch suffer from limitations inherent to their general-purpose design. Because they operate over arbitrary programs, it is often impossible to determine which sub-expression to update when an output is manipulated, necessitating heuristic disambiguation, which can lead to unintuitive outputs in cases of ambiguous edits. Ambiguity is difficult to avoid in bidirectional systems: transformations are often inherently lossy or ambiguous, the likelihood of which increases with the surface area of the target domain.
To address this, we propose an approach which restricts the domain. Specifically, we design and implement a bidirectional drawing system that both defines and constrains drawings to compositions of *epicycles*. In this domain, a curve is defined as the trace of a chain of rotating circles, where each circle is parameterised by a unique radius, frequency, and phase. These circles are created with a Domain Specific Language (DSL) embedded in Haskell. Crucially, the system makes the boundary between the program structure and the visual output thin: users can directly manipulate the parameters of the component circles on the canvas. This, unlike Sketch-n-Sketch, provides an unambiguous link between graphical edits and source transformations, because the user effectively directly interacts with a different representation of the source Abstract Syntax Tree (AST), making the mapping from canvas to code more explicit. While some residual ambiguity remains when back-propagating changes through shared variable expressions, we demonstrate that our domain restriction allows us to resolve this. We compare a heuristic-based approach, against a more recent delta-based semantics, comparing their effectiveness at resolving ambiguity, and providing a predictable user experience.

> The high-level objective of this project is to investigate the theory, and practicality and advantages of domain-specific bidirectional programming by designing, implementing, and formally evaluating a bidirectional epicycle drawing system. More concretely, our specific aims are:
>
> 1. Review existing literature on bidirectional programming and trace-based evaluation to identify best practices for designing bidirectional systems, especially in the creative coding field.
> 2. Implement a bidirectional DSL for creating art as compositions of epicycles, featuring a synchronised, bidirectional graphical interface for editing both views.

[1] https://tidalcycles.org/

3. Compare different approaches for bidirectional programming using our DSL as a simple but practical case study
4. Formalise the semantics of the DSL and prove core correctness properties to ensure predictable and reasonable behaviour
5. Evaluate our DSL's effectiveness, considering both theoretical properties and creative expressivity.

Chapter 2

Background

2.1 Creative Coding and Direct Manipulation

Creative Coding refers to the practice of using code to create something *expressive*, rather than functional. This typically manifests as writing computer programs that generate or edit visual or auditory art. The 'Media Computation' Curriculum [7], pioneered the usage of creative coding for pedagogy, which has continued to the present day, with notable examples like Processing [17], a widely used open source framework for visual creative coding, used in introductory computing courses at universities worldwide. Another famous example is TidalCycles [13], a Haskell DSL for 'live coding' (a development paradigm where the user iterates on code for a program which continually runs, seamlessly updating whenever changes are made) by combining musical patterns. The common pattern among these designs, whether for the purposes of pedagogy or artistic expressivity, is that they aim to shrink the 'abstraction gap' between code and observable behaviour. They all provide high level, declarative interfaces for describing the artistic intention, rather than making the user focus on low-level details.

On the other side of the creative software spectrum is Direct Manipulation[18] – working with computer systems without writing code. In the artistic sector, this specifically refers to using tools like Adobe Illustrator, Figma, or Blender. The benefits and drawbacks of these systems are fairly intuitive – there is no abstraction gap between the way the user interacts with the system and the product they create, however this usually comes at the drawback of lacking powerful scripting or automation tools for editing the work.

The dichotomy of these two paradigms forces digital artists into choosing a compromise: sacrificing the intuitive, immediate feedback of direct manipulation for the composability and abstraction power of creative coding, or vice versa. Bridging this gap and designing systems that can combine the two has become a desirable goal of programming language research, leading to the creation of *bidirectional creative coding environments* such as Sketch-n-Sketch [8]. These aim to offer the best of both worlds by allowing users to programmatically define visual outputs, while simultaneously being able to directly manipulate the rendered canvas, which seamlessly updates the underlying source code.

2.2 The View-Update Problem

Researchers identified that creating these bidirectional creative coding systems are essentially solving the *View-Update Problem*[6], a problem that originates from database theory. In essence, the problem involves a database D and "views" V of D – representations of the data transformed in some way. The problem arises when we make these views editable: how do we map the changes from the edited view back to changes in the original database? To define this more formally, if we produce view V from some mapping $f : D \to V$, and then modify the view, producing some updated V', to what extent can we produce an update $g : D \to D'$ such that $f(D') = V'$?

When f is a bijection, the mapping is trivial: we simply apply $f^{-1}(V')$ to recover D'. However, this is rarely the case. Consider for example, a contrived case where D contains two numeric columns x and y. If we then produce a view V by summing the rows $f(x, y) = x + y$, editing an entry in this view does not have an objective way of applying that edit back to D to get D'. If $x = 6, y = 4$, $z = x + y = 10$, editing $z = 12$ produces infinitely many 'repairs' to D: we could hold one value constant and apply the entire delta to the other, we could distribute the delta evenly, or even make one value negative (e.g.

$x = 20, y = -8$). There is no universally correct way to apply the delta without more direction from the user.

When applied to creative coding, this ambiguity explodes in complexity. If a user defines a function to generate a curve, for example as some combination of trigonometric functions, and then clicks and drags a point of that curve, moving it 10 pixels to the right, how should the system update the underlying source code? It could modify some global scaling modifier, alter a variable shared in multiple expressions, or simply add a constant offset to that specific point.

To resolve this ambiguity, computer science literature has developed several theoretical frameworks designed to constrain the search space of updates. The general solutions fall into a few broad categories:

2.2.1 Lenses

Rather than trying to invert arbitrary functions, we instead try to build bidirectional programs by composing *lenses*[5], a pair of two functions `get` and `put`. These functions retrieve the current value, and perform backwards update respectively, and importantly, are expected to follow strict algebraic laws which we will visit in more detail later. By constructing larger programs by combining these primitives, the system guarantees reliable backward propagation. The biggest drawback of lenses is that a lens inherently can only focus on one part of the program at once. When applied to ASTs, a lens can only focus on a single node. When back-distributing edits through a program structure, lenses cannot reason about the entire topology of the program, instead having to operate purely on the shape of the data.

2.2.2 Cost-based and heuristic search

Lenses are not able to effectively represent all program types, for example when programs have complex control flow, lenses start to become overly restrictive. In these situations, it becomes more useful to consider the whole search space, and apply various cost functions to rank the different possible edits. These cost functions are arbitrary and should be chosen depending on the domain and task. For example, a cost function might penalise updates that mutate shared variables, or prefer updates that distribute changes as evenly as possible, or rank edits based on an edit distance from the original AST. General purpose bidirectional environments such as *Sketch-n-Sketch*[8] tend to use this approach. While effective, these systems can often sacrifice predictability in complex cases – if the solver ranks the user's intent incorrectly, it can introduce inconsistent states or unpredictable AST edits.

2.2.3 Domain Restriction

The final approach to resolving ambiguity (and the primary thesis of this project) is to heavily restrict the capabilities of the forward mapping $f : D \to V$. If the programming domain is sufficiently restricted such that there remains a clear link between the source code and visual output (such as with our epicyclic approach), the mapping from a visual edit back to the AST becomes more explicit and less ambiguous. The tooling has less work to do, and edits remain predictable for the user.

2.3 Bidirectional Programming and Optics

We will now investigate the aforementioned *lenses*, which generalise into *optics*. Formally, a lens l on some source S which 'focuses' on some view V is a bidirectional transformation between S and V. It is typically represented as a pair of functions, a forward transformation $get : S \to V$ and a backwards transformation $put : (S, V) \to S$. These two functions extract the focused view from the source, and update the source given a modified view, respectively.

To ensure predictable behaviour, especially under composition, "well-behaved"[1] lenses are expected to follow a few intuitive laws:

- *GetPut*: $\forall s \in S,\ \text{put}(s, \text{get}(s)) \equiv s$ (Updating a source with its own unmodified view yields the same unmodified source)
- *PutGet*: $\forall s \in S, v \in V,\ \text{get}(\text{put}(s, v)) \equiv v$ (Extracting a view immediately after an update yields the value that was just set)

[1] not all lenses are well-behaved, which is sometimes a useful restriction to relax

- *PutPut*: $\forall s \in S, v_1 \in V, v_2 \in V,\ \text{put}(\text{put}(s, v_1), v_2) \equiv \text{put}(s, v_2)$ (A sequence of updates is solely determined by the final update)

Lenses specifically model one-to-one relationships, that typically being extracting and updating a field within a larger data structure. In practice, bidirectional programming requires reasoning about a wider family of relationships, which leads to the generalisation from lenses to *Optics*. However, lenses begin to struggle under more complex scenarios like aliasing, which motivates more powerful frameworks we will explore later.

2.3.1 Edit Lenses

Standard lenses and their profunctor generalisations are highly effective for manipulating discrete data structures, however they face limitations when applied to more continuous domains which are common in creative coding. A standard lens operates on absolute states – put : $(S, V) \to S$ – and so it requires the entire target view to compute the updated source. When we apply this to creative coding, issues begin to arise: consider a user dragging a point from $(10, 15)$ to $(15, 15)$. A standard lens only sees the final absolute coordinate, and so to push the change through the AST it has to essentially perform inverse kinematics to figure out how the program could evaluate to $(15, 15)$.

To mitigate this, Hofmann et al. created *Edit Lenses*[9] (often referred to as *Delta Lenses*). Rather than replacing the old state with an entirely new, unrelated one, an edit lens operates on *changes* (denoted by Δ). Specifically, we define a monoid of edits over the data structures we want to work with, and then define a homomorphism between these edits. An Edit Lens then translates a view edit (ΔV) to a corresponding source edit (ΔS). We include a hidden state component (a 'complement') to maintain synchronisation between the two structures in the cases of destructive edits.

Returning to the dragged point example, the edit lens redefines the problem: the lens receives the edit in the form of a translation vector $\Delta = (5, 0)$. Pushing this back through the AST becomes far more practical computationally, as the system can simply distribute Δ across the relevant variables or transformations that determine the point, rather than having to guess the user's intent and program structure.

The biggest drawback of this approach is the necessity to define all possible edits as a monoid, which depending on the domain can be very complex, or even impossible. We will revisit this later in the context of our system.

2.3.2 Shared State and Bidirectional Evaluation

As mentioned, lenses fundamentally struggle to reason about expressions that contain shared state or aliasing. Consider, for example, a scenario where a single variable defines parameters for multiple drawn objects (e.g., `let r = 10 in ...`). Static optics can only reason about one structural layer of the AST at a time. An optic operating on a downstream leaf node has no semantic awareness of the layers "above" it, so it cannot inherently modify the bound variable of r. We might think to include this as an extra variable that we thread through, but this tends to break the standard model of optic composition and the well-behaved lens laws. While edit lenses do carry an extra state, this state is purely *structural*. In this context, while an edit lens would be aware of the entire AST, the lens at a given node does not know *where* it is – just like normal optics, they are still purely local.

This leads us to an alternative model of bidirectional evaluation proposed by Mayer et al.[12]. Rather than taking a little-step approach like optics, this takes a big-step approach to defining a bidirectional evaluation semantics based on the entire AST. We define an explicit semantics for our language in both directions: evaluation as $(E \vdash e) \Rightarrow v$ (expression e in environment E evaluates to value v) and evaluation *updates* as $(E \vdash e) \Leftarrow v' \rightsquigarrow (E' \vdash e')$ (when updating its output value to v', environment E and expression e updates to E' and e' respectively).

The biggest advantage of this approach is that our bidirectional semantics can now reason about the AST from a 'zoomed-out' perspective, which we will evaluate the success of later.

2.4 Epicyclic Geometry and the Fourier Transform

We begin this section by defining an *epicycle*, a primitive which will be fundamental to our DSL's design. Traditionally, an epicycle describes a small circle[2] whose centre moves around the circumference of a

[2] even more traditionally, a planet

larger circle. We define epicycles with three main parameters:

- **Radius** r: The length of the epicycle's 'arm'
- **Frequency** ω: The rotational speed of the epicycle (measured in cycles per some arbitrary time unit).
- **Phase** ϕ: The starting angle/offset of the arm relative to the epicycle's start point

To approximate the appearance of code in our DSL, we will often use the notation epicycle(r, ω, ϕ) to describe an epicycle with the given parameters.

We note that by definition, there is an intuitive notion of 'appending' epicycles, combining them into a chain of n epicycles, where the position of the nth cycle can be determined as the cumulative sum of the positions of all previous sections.

An epicycle e_k can be formally defined by a parametric equation over a time parameter t:

$$e_k(t) = (r_k \cos(\omega_k t + \phi_k), r_k \sin(\omega_k t + \phi_k)) \tag{2.1}$$

From Euler's Formula, this can also be expressed as a complex number:

$$e_k(t) = r_k e^{i(\omega_k t + \phi_k)} \tag{2.2}$$

We can then define an epicycle *chain* of length N as the pointwise sum of these epicycles:

$$f(t) = \sum_{k=1}^{N} r_k e^{i(\omega_k t + \phi_k)} \tag{2.3}$$

These form the basis of curves commonly known as *spirographs*[3], but more formally, are identical to finite complex Fourier series / trigonometric polynomials.

To formalise this concept of combining epicycles, we consider the set $\mathcal{F}$ of all finite sums of epicycles. We observe that this set forms a commutative monoid $(\mathcal{F}, +, 0)$ under function addition:

- **Identity**: The constant zero function $f(t) = 0$
- **Associativity**: Inherited from complex addition
- **Closure**: The sum of two finite epicycle chains is itself a finite epicycle chain

2.4.1 The Discrete Fourier Transform

While direct manipulation allows users to redefine existing programmatic structures, drawing a complex shape purely with code or manual arm placement is unintuitive and inefficient. To provide users a manual way of turning a drawing into code, we lean on the *Discrete Fourier Transform* as a program generator.

We can do this thanks to the Fourier Theorem, which states that any continuous periodic function can be expressed as a sum of sine and cosine terms, in other words, as a chain of epicycles[16]. When a user draws freehand on the canvas, we sample the path of their mouse as an ordered sequence of N discrete 2D coordinates. By treating these coordinates as points in the complex plane $z_n = x_n + iy_n$, we can apply the DFT to compute the corresponding complex coefficients:

$$C_k = \frac{1}{N} \sum_{n=0}^{N-1} z_n e^{-i\frac{2\pi}{N}kn} \tag{2.4}$$

Each coefficient C_k directly provides the parameters to be turned into an epicycle node in our AST:

- Radius is equal to $|C_k|$
- Frequency is equal to k
- Phase is equal to $\arg(C_k)$

We can then use a *pretty-printer* to form a valid DSL expression for the epicycle chain, which the user can then copy to their program, and edit either programmatically or via direct manipulation.

[3] Mathematically known as hypotrochoids and epitrochoids

2.5 Embedded DSLs in Haskell

A *Domain Specific Language* (DSL) refers to a programming language which, unlike a general purpose programming language, is intended to be used for a specific application domain. Common examples of these are Structured Query Language (SQL) for relational database operations, or HyperText Markup Language (HTML) for describing web pages. DSLs usually focus on providing better abstractions for working with the target domain, and as such, are often (though not always) more declarative than general purpose languages. They are also often much more constrained – sacrificing Turing completeness, for example, is often a desirable design choice for DSLs.

The biggest downside of DSLs usually comes from their inherent nature of being a new programming language: new tooling has to be created to work with the DSL, and programmers have to learn a new language to work with the domain.

This motivates the technique of *embedding*, the design pattern of implementing a Domain Specific Language within some other 'host' language. This covers a wide variety of applications: something as simple as Java's Stream API could be considered an embedded DSL, and on the other end of the spectrum are powerful, complete programming languages, such as Clash, a hardware description language implemented within Haskell.

The benefits of embedding are clear: the host language's tooling can be reused, the syntax is more familiar to programmers, and the DSL gets interoperability with its host language 'for free'.

We use the terms *deep* and *shallow* to distinguish between the two main approaches of designing and implementing embedded DSLs.

2.5.1 Shallow Embedding

With shallow embedding, the DSL's constructs map directly to the semantics of the host language. There is no intermediate data structure, evaluating a DSL expression simply executes some function in the host language. This approach is trivial to implement, benefits from the host compiler's native optimisations, and in some cases is the only practical approach. A compelling example of a shallowly embedded DSL are most *parser combinator* libraries, which still represent parsers as functions under the hood, but allow using operators like `<|>` and `sepBy` to combine these functions in a declarative way.

The drawbacks of shallow embedding are that we lose flexibility and any capability for introspection. There can only be one semantics for our DSL, and implementing any sort of program analysis (such as a pretty-printer) is extremely difficult or impossible. This makes shallowly embedded DSLs usually ill-suited for bidirectional programming, as the necessary program introspections such as provenance tracking become impossible.

2.5.2 Deep Embedding

On the other hand, deeply embedded DSLs build a representation of the program as an AST first, before executing it. The DSL code is first transformed to pure data, and is only executed when passed to a separate interpreter. This allows the DSL implementer to perform program optimisation and introspection, along with allowing multiple interpretations/semantics of a program (e.g. pretty printing, interpreting, compiling, symbolic execution, etc.). The primary shortcoming of this approach is that its design constraints can cause loss of expressivity. To understand the cause of this, we can look at the example of *binders*

When we implement lambdas or any other bindings in DSLs, it is highly desirable to re-use the host language's native binding syntax. In Haskell, this approach is known as *Higher-Order Abstract Syntax (HOAS)*[15]. Using HOAS, a DSL term for a function might look like `Lam (\x -> Add x (Lit 1))`. This means our DSL inherits the host language's lexical scoping, type checking, etc, 'for free'.

However, this convenience introduces the fatal flaw for deep embedding: the lambda expression, unlike the rest of our DSL, is fully opaque. We have essentially returned to using a shallow embedding for binders – when our AST traversal algorithm encounters a `Lam`, it cannot introspect the Haskell function's contents, without providing some value to the function. This is an acceptable solution in some cases, but is often severely limiting.

As such, many DSLs reject the 'overpowered' HOAS and turn to a simpler design pattern: *First Order Abstract Syntax (FOAS)*[10]. As the name suggests, this representation encodes our language using only first order data: we replace shallowly embedded lambdas with manual references to binders (for example as `String`s or `Int`s). A common approach to this implementation is the use of *De Bruijn Indices*[4], where instead of using named variables, we represent bindings by natural numbers that reference a variable by

its distance (in terms of binding depth) from the variable's usage and its corresponding binder. With FOAS and De Bruijn indices, our previous example DSL term might now look like this: `Lam (Add (Var 0) (Lit 1))`.

FOAS provides full introspection power for our language, but at the devastating loss of expressivity and safety – there is now no longer anything stopping a user from mistakenly referencing a non-existent variable (this can be mitigated by threading an environment through the type, a technique we will employ later), and we can no longer use the host language's tooling to provide useful utilities such as autocomplete for variables in scope. Furthermore, this representation tends to be much harder for programmers to read and reason about.

2.5.3 Embedding by Unembedding

We are left with a dichotomy: HOAS provides the ideal developer experience but is opaque to structural analysis. FOAS provides perfect transparency but a hostile developer experience. This has motivated various research projects to try and resolve the tension. In particular, we turn to *Embedding by Unembedding*[11], a technique developed by , including other members of the Bristol Programming Languages Research Group (which includes my supervisor). This library provides a framework for defining a 'Frontend' DSL using HOAS, and then 'unembedding' it into a FOAS representation using De Bruijn indices. Furthermore, it treats this compilation process as just one example of an 'advanced semantics' – making it especially suitable for languages whose semantic domains are not "naturally expressed as functions... [such as] bidirectional languages"[11].

Tagless Final Representation

The Embedding by Unembedding approach requires a specific design pattern for DSLs known as *Tagless Final Style*[1]. This involves using type classes as the main interface for the DSL, rather than concrete data types.

For example, the interface for the DSL (in HOAS form) we have used in previous examples might be defined in a traditional style using a *Generalised Algebraic Data Type (GADT)*:

```
data DSL a where
^^ILit :: Int -> DSL Int
^^IAdd :: DSL Int -> DSL Int -> DSL Int
^^ILam :: (DSL x -> DSL y) -> DSL (x -> y)
```

In tagless final style, we abstract the shape of our interface into a type class parameterised over the concrete representation `repr`:

```
class DSL (repr :: Type -> Type) where
^^Ilit :: Int -> repr Int
^^Iadd :: repr Int -> repr Int -> repr Int
^^Ilam :: (repr x -> repr y) -> repr (x -> y)
```

Because the syntax is defined as a class rather than concrete data, the DSL can support multiple interpretations simply by defining different class instances. Therefore, to solve the 'binder opacity problem', . allow defining the compilation from HOAS to FOAS as an instance of the class, providing a framework for doing so in a concise and type-safe manner. This provides the ideal architecture for our bidirectional DSL: expressive in the frontend with ergonomic HOAS, while still allowing the bidirectional evaluator to operate on transparent, De Bruijn-indexed FOAS data.

Chapter 3

Project Execution

3.1 Architectural Evolution and Scope

The final design of our DSL is the result of a number of iterative designs. We will begin by describing our initial approach, so as to understand the design of our final product. Initially, the project targeted a much wider domain: allowing any arbitrary parametric mathematical function to be plotted and directly manipulated with back-provenance. For example, an early prototype allowed users to define shapes such as ellipses as parametric, *open* terms:

```
ellipse :: Curve (Value, Value) Value (Value, Value)
ellipse =
  \(param1, param2) t ->
    pair
      (mul param1 (cos t))
      (mul param2 (sin t))
```

The `Curve` type is parametrised over the type of its open (external) parameters `(Value, Value)`, its internal parameter t of type `Value`, and its output of type `(Value, Value)`. This allowed a clear distinction between parameters that could be edited by direct manipulation, and ones that could not. The system would then take this curve definition and produce an animation of its trace that could be directly manipulated.

The `Value` type defines a simple symbolic algebra system which handles numerical values as their expressions in an AST, rather than simple floating point values. This was to avoid destructive back-propagation, where a readable expression like `sin(x) + cos(y)` might be replaced with some long, difficult to read, and no longer parametric numeric value. A basic simplification system was also included to try and prevent overly complex expressions.

Back-propagation in this system was done with lenses, where every DSL element had its own lens interpretation thanks to the tagless final style. Unfortunately, the semantics of this approach led to expression blow-up when back-propagating over complex expressions. It is worth taking time to formalise and discuss this approach to understand its failures.

To understand the point of failure, we briefly define the semantics for a very simplified version: open (editable) parameters x, number literals n, and a binary multiplication node $e_1 \cdot e_2$:

We let Γ be the environment of variables to values, $\mathcal{E}$ to be the syntactic domain of AST expressions (i.e. values of type `Value`), and $\mathcal{V}$ to be the semantic domain of AST expressions.

We also define a (well-behaved) lens as a pair of functions $\mathrm{Get}_e : \Gamma \to \mathcal{V}$, $\mathrm{Put}_e : \Gamma \times \mathcal{V} \to \Gamma$, for each expression $e \in \mathcal{E}$.

Literals are always held constant, and so their semantics are trivial:

$$\mathrm{Get}(n, \sigma) = n, \quad \mathrm{Put}(n, \sigma, v) = \sigma$$

For a parameter x, we have

$$\mathrm{Get}(x, \sigma) = \sigma(x), \quad \mathrm{Put}(x, \sigma, v) = \sigma[x \mapsto v]$$

In other words, we either read the value of x from the environment σ, or we update the environment to set x to a new value v.

The lens semantics for multiplication, the main source of the blow-up, was defined as such: for a binary multiplication node $e_1 \cdot e_2$, the semantics attempted to back-propagate the delta evenly across e_1 and e_2. Specifically, given a target value v', the *Put* semantics were defined as:

$$\text{Put}(e_1 \cdot e_2, \sigma, v') = \text{Put}(e_2, \text{Put}(e_1, \sigma, v_1'), v_2')$$

where intermediate target values v_1' and v_2' are defined as

$$v_1 = \text{Get}(e_1, \sigma), v_2 = \text{Get}(e_2, \sigma)$$
$$r = \sqrt{\frac{v'}{v_1 \cdot v_2}}$$
$$v_1' = v_1 \cdot r, v_2' = v_2 \cdot r$$

We can observe the blowup by tracing these semantics over a simplified case: $e = \text{amp} \cdot cos(t)$. Assume the user drags a point of this curve across the canvas. At this point, time is paused at time t_0 and we have some environment $\sigma_0 = \text{amp} \mapsto a_0$.

After the first frame of dragging, the user moves the mouse to some new target value v_1. The lens therefore calculates the ratio

$$r_1 = \sqrt{\frac{v_1}{a_0 \cdot \cos(t_0)}}$$

, and produces two new target values $v_1' = a_0 \cdot r_1$ and $v_2' = \cos(t_0) \cdot r_1$, which it then pushes back through the AST.

The lens applies this function to both operands, but crucially, because $\mathcal{V}$ is a domain of *symbolic AST nodes* rather than evaluated floats, the multiplication $v_1 \cdot r_1$ cannot always be simplified to a single number. The environment therefore updates to

$$\sigma_1(\text{amp}) = a_0 \cdot \sqrt{\frac{v_1}{a_0 \cdot \cos(t_0)}}$$

Upon the second drag iteration, a new target value v_2 is produced. The process repeats, producing a new

$$r_2 = \sqrt{\frac{v_2}{\sigma_1(\text{amp}) \cdot \cos(t_0)}}$$

forming a new environment

$$\sigma_2(\text{amp}) = \sigma_1(\text{amp}) \cdot \sqrt{\frac{v_2}{\sigma_1(\text{amp}) \cdot \cos(t_0)}}$$

It is clear to see how this causes an unbounded blow-up over time. While some simple cases of this could be prevented by aggressively trying to simplify `Value`s, this is computationally expensive and does not solve the problem in the general case.

Our first attempt to remedy this involved making some lenses *constraint-aware*. We observed that under the constraints of the system, $\cos(t_0)$ is a constant. Due to the semantics of constants being uneditable, the multiplication lens cannot blindly assume that splitting the change evenly across both operands will always succeed – in this case, pushing back the change to $\cos(t_0)$ will not change it at all. We redefined the semantics to fallback to a simple linear solve in the case that one of the operands is unchanged by the attempted update: the lens first attempts the even split, then checks if either operand is unchanged by the attempted update, and if so, applies the entire change to the other operand instead.

Under these revised semantics, the first drag iteration notices that $\cos(t_0)$ is unchanged, and so applies the entire change to amp instead, producing a new, correct environment

$$\sigma_1(\text{amp}) = \frac{v_1}{\cos(t_0)} = v_1 \cdot \frac{1}{\cos(t_0)}$$

Following this, we can see the second drag iteration makes the same observation, and produces a new environment

$$\sigma_2(\text{amp}) = \frac{v_2}{\cos(t_0)}$$

We seem to have solved the blow-up problem for some simple curves, however this reveals a deeper issue with the lens semantics

This works, but we can now produce a new failing counterexample: $e = (\mathrm{a} \cdot \mathrm{b}, \mathrm{a} \cdot \mathrm{b})$. This defines a curve where both coordinates are defined by the same expression.

To understand why this fails, we need to slightly extend our semantics to also include pairs:

$$\mathrm{Get}((e_1, e_2), \sigma) = (\mathrm{Get}(e_1, \sigma), \mathrm{Get}(e_2, \sigma))$$
$$\mathrm{Put}((e_1, e_2), \sigma, (v_1', v_2')) = \mathrm{Put}(e_2, \mathrm{Put}(e_1, \sigma, v_1'), v_2')$$

Consider our example $e = (a \cdot b, a \cdot b)$ under some environment $\sigma_0 = \{a \mapsto 2, b \mapsto 3\}$. This initially evaluates to $v = (6, 6)$. Suppose we attempt to update this point to a new target $v' = (7, 7)$

The lens first attempts to satisfy the X coordinate constraint by updating $e_1 = a \cdot b$ to reach $v_x' = 7$. As established before, this applies a symmetric ratio $r_x = \sqrt{\frac{7}{6}}$ to both operands, yielding an intermediate environment $\sigma_x = \{a \mapsto 2\sqrt{\frac{7}{6}}, b \mapsto 3\sqrt{\frac{7}{6}}\}$

The lens then proceeds trying to satisfy the Y coordinate constraint, updating $e_2 = a \cdot b$ to reach $v_y' = 7$. Crucially, because lenses are strictly local, the Y update has no awareness that a and b were just updated, and so it applies the same process to σ_x, producing a new final updated environment

$$\sigma_1(a) = \sigma_x(a) \cdot \sqrt{\frac{7}{\sigma_x(a) \cdot \sigma_x(b)}}$$

The AST size effectively doubles with every iteration, resulting in a truly exponential blow-up.

It is tempting to suggest that these failures can still be patched – for instance, we might think to add checks to the pair lens to verify if an update to e_1 incidentally satisfies the constraints for e_2, and if so, skip the update to e_2 entirely. However, these mitigations are not only 'whack-a-mole' solutions to specific cases, but patching them fundamentally breaks the abstraction of lenses. Lenses are designed to be strictly local and compositional – to truly solve these problems, we need global awareness, constraint ranking, and backtracking. Attempting to add these mechanisms to a lens effectively degrades it into a poorly designed constraint solver. Therefore, the architecture was not merely buggy, but fundamentally misaligned with the domain requirements of multi-variable symbolic manipulation.

3.2 Domain Restriction: Motivation

It is clear that our initial approach of making a general purpose bidirectional programming for arbitrary parametric curves was not working – the question of whether it is theoretically possible is still open and out of scope for this project.

We have analysed the weaknesses of allowing arbitrary computation from a formal perspective, and will now briefly analyse it from a *UX / HCI* perspective. We return to the seminal example of bidirectional creative coding environments: *Sketch-n-Sketch*[2, 8], a system for bidirectional programming of SVG graphics.

Sketch-n-Sketch achieves a great deal of expressivity by supporting arbitrary functional drawing programs. The DSL is extremely expressive, supporting functions, pattern matching, and a large library of primitive shapes that compose. However, the large surface area of this language also inherently makes the View-Update problem ambiguous in many cases.

Sketch-n-Sketch largely accepts this as a limitation which it pushes to the user: *"There are often multiple valid program changes associated with a specific user action. To resolve ambiguity, our system displays code and output differences and asks the user to choose"*, though it often applies a heuristic behind-the-scenes to try and pick the most likely candidate. Furthermore, Sketch-n-Sketch pushes the ambiguity problem to the *language*, introducing a 'freeze operator' `!`, which allows users to mark certain constants as uneditable.

It seems that when the domain is sufficiently complex, structural ambiguity must be resolved by either guessing the user's intent via opaque heuristics, or by pushing the problem to the user via some form of UI. Both of these solutions are suboptimal: the former can lead to unpredictable edits, while the latter can lead to a less 'fluid' user experience.

Our initial prototype failed because rather than designing around the geometry and visual domain, we designed the DSL around a general purpose symbolic algebra system, and then tried to patch it with heuristics to make it work for our domain. There is simply not enough structure in the domain to make the View-Update problem tractable. We think back to our failing example of the spirograph, and our

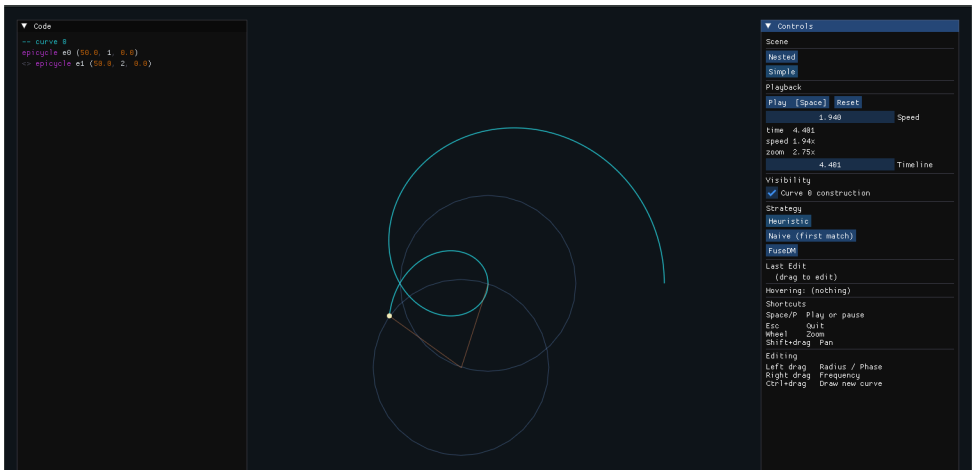


Figure 3.1: The main UI of our system, showing the code pane on the left, the rendered curve in the middle, and extra controls on the right.

new design elegantly falls into place: the spirograph is a curve defined by a chain of epicycles, which inherently have a more well defined mathematical structure. Each epicycle is independent, and has a clear mapping between its source code and the geometry it produces. Furthermore, by using the Discrete Fourier Transform, we can model any continuous curve as a chain of epicycles, giving us a clear path to generality and expressivity without sacrificing structure.

3.3 UI and Interaction Model

Our system is formed of two components: an **embedded DSL** in Haskell for defining curves as epicycles, and a **visual editor** to render the curves and allow *direct manipulation*, which back-propagates into new DSL terms.

The full UI of our system shows a window with 3 panes. The first pane shows the pretty-printed code used to generate the curve. The second pane shows the rendered curve, which can be directly manipulated by clicking and dragging various parts. The final pane has some extra controls for a better user experience, such as being able to change the animation speed, scrub through time values t manually, change the backward semantics used, and toggle the visibility of the curve's construction. Figure 3.1 shows a screenshot of the main UI.

We solve the ambiguity problem of other direct manipulation systems like Sketch-n-Sketch by changing the interaction model. Rather than interacting with the final traced curve, which inherently creates ambiguity, we explicitly render the *construction* of the curve – i.e. the epicycles that make up the curve – and allow direct manipulation of these instead. Every user interaction therefore maps unambiguously to a specific parameter of a specific epicycle. Therefore, while it may not always be unambiguous *how* to update it (as we will see soon), it is always unambiguous *which* sub-expression needs to be updated.

Users can hover over either the circumference, or the 'arm' of the epicycle, which correspond to the radius and phase parameters respectively. Clicking and dragging while hovering over one of these components triggers an update towards the new value the user is dragging 'towards'. The user can also right-click and drag anywhere on the epicycle, which does the same for the *frequency* – unlike the other parameters, this does not have a direct visual component. The frequency is also restricted to being an integer in our system, and so the dragging jumps between integer values, rather than being a continuous update.

The UI shows the construction of the curve, visualising the epicycles that make up the final trace. This fulfills the purpose we set out of making the boundary between AST and visual output unambiguous and direct. We also take care to emphasise this throughout the user experience. When hovering on a particular part of the curve's construction, it is highlighted in yellow, and the corresponding `epicycle` term is syntax highlighted in a different colour. The specific parameter expression is highlighted in magenta to show exactly what is being edited, along with a textual description of the highlighted element on the right pane. This is shown in fig. 3.2.

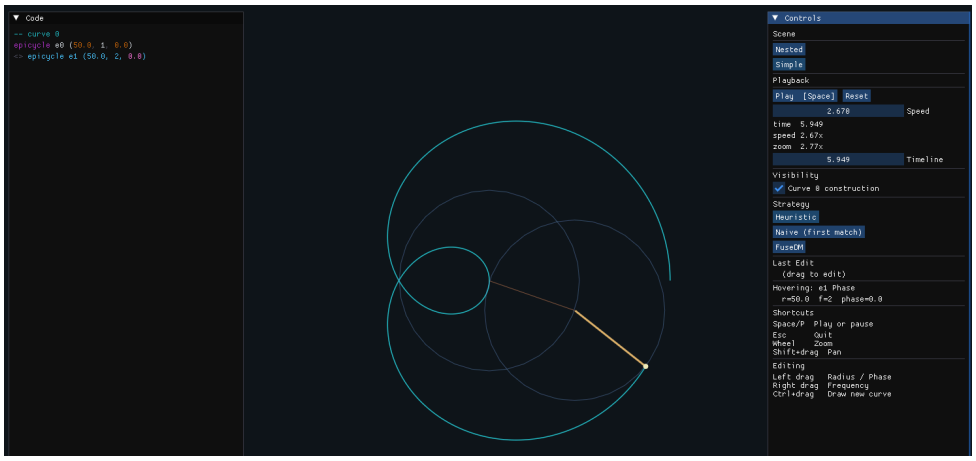


Figure 3.2: The UI clearly shows the connection between the code and the visual output, highlighting the relevant code and construction element when hovering. In this example, we are hovering over the phase parameter (the arm) of the second epicycle.

3.4 DSL Design

3.4.1 Syntax

We present the surface syntax of our DSL in Backus-Naur Form (BNF) at a high level.

Expr	e	$::=$	n	literal number $(n \in \mathbb{Q})$
		$\mid$	x	parameter or variable $(x \in \Sigma)$
		$\mid$	$e_1 + e_2$	addition
		$\mid$	$e_1 \times e_2$	multiplication
		$\mid$	e^{-1}	reciprocal
		$\mid$	let $x = e_1$ in o	numeric let-binding
Curve	c	$::=$	$\text{epicycle}(e_r, e_\omega, e_\phi)$	Epicycle definition
		$\mid$	$c_1 \oplus c_2$	Curve concatenation
CurveOrExpr	o	$::=$	$e \mid c$	

We define our DSL as two separate sub-languages: a language of numeric expressions, and a language of curves. This provides a clear separation of concerns, simplifying the language for the end user but also simplifying our implementation.

One important design choice is that let-bindings are only allowed to bind numeric values, and not curves. This makes expressions like let $c = \text{epicycle}(e_r, e_\omega, e_\phi)$ in $c \oplus c$ unrepresentable. This prevents geometric aliasing, which causes our DSL to represent a Directed Acyclic Graph (DAG) rather than a strict tree. This would cause backwards solving to become exponentially more complex with far more ambiguous updates.

For similar reasons, our language is deliberately minimal. We include only the minimum viable constructs for a reasonable level of expressivity. The notable omissions of subtraction and division can be derived from the existing constructs.

Canonicalisation

From a purely mathematical perspective, a chain of epicycles can be canonicalised: any two epicycles with identical frequencies can be algebraically merged into a single epicycle by combining their radii and phases as complex vectors. However, we reject this in our system – in the context of bidirectional creative coding, automatically merging structurally distinct AST nodes violates the principle of preserving the programmer's intent. Epicycles may be defined with identical frequencies to logically group sections of source code, or for prototyping. By keeping the AST unadulterated, we maintain an intuitive 1:1 mapping between the source code and visual domain. We emphasise that this lack of canonicalisation does *not* reintroduce ambiguity into the View-Update problem: because our system exposes the *construction* of the curve rather than just the final trace, the user interaction is always targeted and uniquely identifiable to the exact component epicycle.

3.4.2 Statics

To formalise the statics of our language, we define a closed universe of types available in the DSL. We distinguish between numeric types η, which may be bound to variables, and the general universe of types τ:

$$\begin{array}{rcl} \eta & ::= & \mathbb{Z} \\ & | & \mathbb{Q} \\ \tau & ::= & \eta \\ & | & \text{Curve} \end{array}$$

Our statics are defined by the typing relation $\Gamma \vdash e : \tau$, where Γ is an environment mapping variables to types.

$$\frac{n \in \mathbb{Z}}{\Gamma \vdash n : \mathbb{Z}}\ \text{T-LitInt} \qquad \frac{n \in \mathbb{Q}}{\Gamma \vdash n : \mathbb{Q}}\ \text{T-LitRat} \qquad \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma \vdash e_1 : \eta \qquad \Gamma \vdash e_2 : \eta}{\Gamma \vdash e_1 + e_2 : \eta}\ \text{T-Add}$$

$$\frac{\Gamma \vdash e_1 : \eta \qquad \Gamma \vdash e_2 : \eta}{\Gamma \vdash e_1 \times e_2 : \eta}\ \text{T-Mult} \qquad \frac{\Gamma \vdash e : \eta}{\Gamma \vdash e^{-1} : \eta}\ \text{T-Recip} \qquad \frac{\Gamma \vdash e_1 : \eta \qquad \Gamma, x : \eta \vdash o : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } o : \tau}\ \text{T-Let}$$

$$\frac{\Gamma \vdash e_r : \eta \qquad \Gamma \vdash e_\omega : \mathbb{Z} \qquad \Gamma \vdash e_\phi : \eta}{\Gamma \vdash \text{epicycle}(e_r, e_\omega, e_\phi) : \text{Curve}}\ \text{T-Epicycle}$$

$$\frac{\Gamma \vdash c_1 : \text{Curve} \qquad \Gamma \vdash c_2 : \text{Curve}}{\Gamma \vdash c_1 \oplus c_2 : \text{Curve}}\ \text{T-Concatenate}$$

Normal Forms

The decision to restrict let-bindings to only bind numeric values allows us to define a clear normal form for our DSL. Because curves cannot be bound to variables, no curve expression can ever be nested inside the evaluation scope of a numeric expression. This leads the language to separate into 3 *non*-mutually recursive layers: purely numeric expressions, variable binders, and curve compositions. Consequently, any valid AST can be partitioned into a Normal Form, where let bindings are 'floated' to the root of the program without affecting the evaluation of the expression.

Definition 1 (Let-Normal Form)**.** *An expression P^* is in Normal Form when of the form*

$$P^* = \textit{let } x_1 = e_1 \textit{ in let } x_2 = e_2 \textit{ in } \ldots \textit{ let } x_n = e_n \textit{ in } C$$

where each e_i is a purely numeric `Expr` *containing no further* `let` *bindings, and C is a* `Curve`

Theorem 1 (Normalisation)**.** *For any well-typed program $\sigma \vdash P$: Curve, there exists a program P^* in let-normal form such that P and P^* are semantically equivalent*

Proof. We proceed by structural induction over the syntax of P. We first define the floating operation as a transformation that commutes let-bindings outwards of every syntax rule.

Numeric Constructors: For any binary numeric operation $\odot \in \{+, \times\}$:

$$(\text{let } x = e_1 \text{ in } e_2) \odot e_3 \equiv \text{let } x = e_1 \text{ in } (e_2 \odot e_3)$$
$$e_1 \odot (\text{let } x = e_2 \text{ in } e_3) \equiv \text{let } x = e_2 \text{ in } (e_1 \odot e_3)$$

A similar rule applies for the unary reciprocal operator which we omit for brevity.

Curve Constructors: By the T-Let typing rule, variables can only bind numeric types η. Therefore, a `let` can never bind a `Curve`, but may appear inside one of its parameters. We can lift these bindings outward:

$$\text{epicycle}(\text{let } x = e_1 \text{ in } e_r, e_\omega, e_\phi) \equiv \text{let } x = e_1 \text{ in epicycle}(e_r, e_\omega, e_\phi)$$
$$c_1 \oplus (\text{let } x = e_1 \text{ in } o) \equiv \text{let } x = e_1 \text{ in } (c_1 \oplus o)$$

The cases for other curve parameters e_ω and e_ϕ are similar.

Base Cases: Literals and variable references cannot contain any `let` bindings and so are trivially in normal form.

Inductive Case: For any expression containing a `let` binding, we repeatedly apply the rewrite rules to push the binding 'up' one level of the AST. Because the AST is finite, and the rewrite rules strictly decrease the depth of the `let` node without duplicating it, this process is guaranteed to terminate.

□

3.4.3 Expressivity

This DSL captures finite epicycle chains whose parameters are determined by closed-form first-order arithmetic with let-bindings. Because all terms are closed, every parameter ultimately reduces to a composition of literals through $+$, $\times$, and $^{-1}$, yielding a concrete value at evaluation time. More formally, the DSL expresses the set of finite trigonometric polynomials of the form

$$f(t) = \sum_{k=1}^{N} c_k \cdot e^{i(\omega_k t + \phi_k)}$$

where each coefficient $c_k \in \mathbb{C}$ is determined by a closed arithmetic expression over $\mathbb{R}$ (or $\mathbb{Z}$ for frequencies).

3.4.4 Implementation

We implement our DSL as a deep embedding in Haskell, using the Embedding by Unembedding approach introduced in section 2.5.2.

Type-Level Architecture

The bipartite type system we have defined is somewhat difficult to express cleanly in Haskell. Typically when embedding DSLs, we index the type of each expression by some type variable: for example `data AST env a where ...`. This however presents issues with implementing our T-Let rule, constraining the type of the bound variable to be a numeric type. We might think to use a custom type class constraint:

```
^^I-- | Bind a variable
^^ILet :: (BindableNum n) => AST env n -> AST (n : env) a -> AST env a
^^I-- | Concatenate two curves
^^IAppend :: AST env Curve -> AST env Curve -> AST env Curve
```

However, Haskell adopts an "open-world" approach to type classes. We cannot assume that `instance BindableNum Curve` won't be defined in the future. This design forces us to write partial functions: some function which evaluates numeric expressions

```
^^IevalExpr :: (BindableNum n) => AST env n -> n
```

still requires including an unsafe case

```
evalExpr (Append a b) = error "unreachable"
```

To ameliorate this, we define a closed-world of *kinds*. Kinds allow categorisation of types, just as types categorise values. For example in Haskell, we say that `Int :: Type`, `Maybe :: Type -> Type`, etc. In the above example, `a :: Type`, which is problematic because it gives no guarantees about the specific types that are allowed. We use GHC's `TypeData` extension to define a new kind `ASTSort`:

```
{-# LANGUAGE TypeData #-}
type data ASTSort = SNum Type | SCurve
```

We can then index our AST by this new kind:

```
data AST env (a :: ASTSort) where
  Let :: (BindableNum n) => AST env (SNum n) -> AST (SNum n : env) a -> AST env a
  Append :: AST env SCurve -> AST env SCurve -> AST env SCurve
```

To bridge the gap between `ASTSort` and runtime values, we use the *singleton pattern*. We define a GADT `SortVal`, which serves as a term-level witness for the `ASTSort` kind:

```
data SortVal (s :: ASTSort) where
  VNum   :: n -> SortVal (SNum n)
  VCurve :: Curve -> SortVal SCurve
```

This establishes a strict 1:1 correspondence between the type-level index and the term-level value. If the index is `SNum n`, the runtime value is structurally guaranteed to be `VNum`, which follows the same logic for `SCurve` and `VCurve`.

By populating our environment with these singletons (`type IEnv = Env SortVal`), pattern matching on the environment yields type equality proofs directly to the compiler.

The type signature for the evaluator now becomes `evalExpr :: AST env (SNum n) -> n`. Because `ASTSort` is closed, GHC can prove that the `SCurve` kind returned by `Append` will never unify with `SNum`, satisfying the exhaustive pattern matching checking without needing explicit unsafe cases.

We still use the `BindableNum` class, which remains necessary but serves a distinct purpose: While `ASTSort` structurally isolates the AST branches, `BindableNum` enforces compile-time constraints, ensuring that nonsensical type combinations (such as attempting to construct an AST node for `SNum String`) are impossible to construct.

However, one partial case remains: `Param`. Although we guarantee *within* our DSL that only numeric types can be bound, there is nothing stopping an expression being defined that references a free variable of type `SCurve`:

```
badExpression :: AST '[SCurve] SCurve
badExpression = Param IZ
```

Trees That Grow

When dealing with Abstract Syntax Trees it is often beneficial to have multiple *stages* of the AST, where each stage has different invariants, or is annotated with extra information. Indeed, for our system, we have two main stages: the initial "parsed" AST, and a second, where every `epicycle` is annotated with a unique identifier. We use the *Trees that Grow*[14] approach to re-use the same AST definition for both stages. This is done by adding an extra type level index to the AST, denoting the stage of the AST, and then using *type families* to define different types for each stage. For the unique identifiers, we define the types like so:

```
-- | Marker for the AST's stage
type data Stage
  = Parsed -- ^ the initial AST, converted from HOAS
  | Numbered -- ^ the AST after numbering

-- | Type family for epicycle "eXtensions"
type family XEpicycle (p :: Stage) where
-- the initial AST has no extra information
  XEpicycle Parsed = ()
  -- the numbered AST has an extra EpicycleId field
  XEpicycle Numbered = EpicycleId


data AST (x :: Stage) (env :: [ASTSort]) (s :: ASTSort) where
  -- | Define an epicycle with radius, frequency, and phase
  Epicycle ::
    XEpicycle x -- ^ extension field
    -> AST x env (SNum Rational) -- ^ radius
    -> AST x env (SNum Int) -- ^ frequency
    -> AST x env (SNum Rational) -- ^ phase
    -> AST x env SCurve
```

We also note that the Trees that Grow approach is not only for *adding* data, but also *removing* it. Indeed, this is the main way in which we enforce the constraint that curves cannot be bound to variables. By making a type family which maps some stages to `Void` (the uninhabited type), we guarantee that ASTs of this pattern cannot be constructed:

```
type family XParam (s :: ASTSort) where
```

```
  XParam SCurve = Void
  XParam (SNum n) = ()

data AST (x :: Stage) (env :: [ASTSort]) (s :: ASTSort) where
  -- | Reference a variable / parameter from the environment
  Param_ :: (XParam s) -> Ix env s -> AST x env s
  -- ...
```

This effectively makes it impossible to write an expression which references a variable of type `SCurve`, and solves the previous problem – even if we have a free variable of type `SCurve` in our environment, we cannot write an expression which references it, so it cannot affect our semantics.

We also define a *Pattern Synonym* for the `Param_` constructor, to hide this implementation detail from the rest of the codebase:

```
pattern Param :: (XParam s ~ ()) => Ix env s -> AST x env s
pattern Param ix = Param_ () ix
{-# COMPLETE Lit, Param, Add, Mul, Recip, Epicycle, Append, Let #-}
```

Adding constraints to Embedding-by-Unembedding

To make this invalid state unrepresentable with Trees that Grow, we had to fork the `embedding-by-unembedding` library to relax its requirements of *parametricity*. Matsuda et al.'s work relies on a common interface that is able to "lift semantics of variables into semantics of terms" – in other words, the *Embedding by Unembedding* approach relies on being able to reference variables in our semantics. This is how the library achieves the seamless conversion from HOAS to De-Bruijn indexed syntax. Specifically, this is done with the `LiftVariables` type class[1], which defines a function `liftVar :: Var sem env a -> sem env a` for an associated type family `Var`. Importantly, this function is required to be parametric in `a`, so we cannot write a case for `SCurve` or block it without violating parametricity. We make a small tweak to the library to work around this, adding an extra associated type family `LiftConstraint sem (a :: k) :: Constraint`, utilising GHC's `ConstraintKinds` extension. We then redefine

```
liftVar :: (LiftConstraint sem a) => Var sem env a -> sem env a
```

We can then define $\texttt{LiftConstraint AST}_s = \texttt{XParam Parsed}_s \sim ()$, to enforce that only numeric types can be lifted into the semantics.

The implementation of this change is short but worth discussion. The `unembedding` library evaluates higher-order syntax by performing structural induction over the environment using a type `TEnv`, essentially a list of `Proxy` types. Since `Proxy` carries no information, it does not carry any evidence of the `LiftConstraint` constraint. We expanded the library's existing `OfLength` type to also carry the constraint evidence, and used that in place of `TEnv` throughout the library.

The rest of the changes were minor and mechanical, replacing `TEnv` with `OfLength` and adding the necessary `LiftConstraint` constraints to the type signatures of the various functions that manipulate the environment, and occasionally converting between `OfLength` and `TEnv` when necessary.

The main trade-off of this approach is that the conversion from HOAS to De-Bruijn syntax has a more complex type signature, and so requires some manual type annotations to aid inference. We have not observed any other significant downsides, and argue that this change maintains the library's original design goals while allowing us to enforce the necessary constraints for our DSL.

Final Implementation

We present the frontend HOAS interface for our DSL, which is what user code is written in. The De-Bruijn indexed AST implementation can be found in the accompanying codebase.

```
type data ASTSort = SNum Type | SCurve

data SortVal (s :: ASTSort) where
  VNum :: n -> SortVal (SNum n)
  VCurve :: Curve -> SortVal SCurve
```

[1] A minor deviation from the original paper published to the reference implementation at https://github.com/kztk-m/EbU

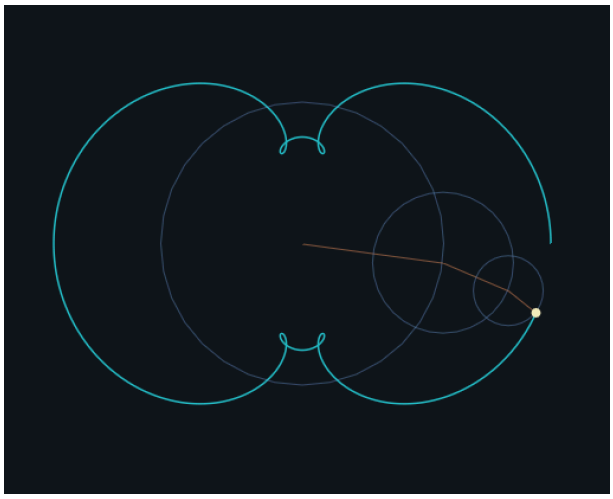


Figure 3.3: The output of an example curve defined in our DSL.

```
class Expr (e :: ASTSort -> Type) where
  lit :: (BindableNum n) => n -> e (SNum n)
  add :: (BindableNum n) => e (SNum n) -> e (SNum n) -> e (SNum n)
  mul :: (BindableNum n) => e (SNum n) -> e (SNum n) -> e (SNum n)
  recip :: (BindableNum n, Fractional n) => e (SNum n) -> e (SNum n)
  let_ :: (Typeable n, BindableNum n) => e (SNum n) -> (e (SNum n) -> e a) -> e a

class (Expr e) => CurveExpr e where
  epicycle :: e (SNum Rational) -> e (SNum Int) -> e (SNum Rational) -> e SCurve
  cappend :: e SCurve -> e SCurve -> e SCurve
```

Each function cleanly maps to part of the syntax of our language, enforcing the statics using type classes and the aformentioned type-level architecture. The `Typeable` constraint is required for bidirectional semantics to compare ASTs, which we will revisit later.

We can now present a complete example of a curve defined in our DSL, which draws a simple shape:

```
curveSimple :: (CurveExpr e) => e SCurve
curveSimple =
    epicycle 120 1 0
        <> epicycle 60 3 0
        <> epicycle 30 5 0
```

Note the use of the general *semigroup* operator `<>`, which we overload as an alias of `cappend`. The graphical output of this is presented in fig. 3.3.

Our DSL's final design shows a clear correspondence between our described syntax and statics, and the real Haskell code. This is one of the main benefits of embedding DSLs – rather than just implementing our language, we *encode* it:

- $\eta \subset$ `SNumType` [2]
- $\tau \subset$ `ASTSort`
- $\Gamma \cong$ `env :: [ASTSort]`
- $x : \tau \in \Gamma \cong$ `Ix env s`
- $\Gamma \vdash e : \tau \cong$ `Expr e => e env s`

[2] As types like `SNum String` are legal to construct, but cannot be used in practice, we cannot say that the two are truly isomorphic

3.5 Semantics

We have discussed the statics and frontend implementation of our DSL, but have not yet covered the semantics. Our implementation aims to evaluate and compare different bidirectional semantics, and so we do not commit to a single *backwards* semantics in this section, however for the most part, the forwards semantics do not vary.

It is useful to abstract the problem domain into a pure data model – while the domain is visual, interactive, and geometric, we can simplify this into a pure data view. To do this, we introduce a semantic visual projection, implemented as the type `VisualScene`. Rather than directly rendering the AST, we define a forward mapping that flattens the hierarchical, algebraic structure of the AST into a purely geometric snapshot of the curve at a given time t.

This type acts as a *view model*, separating concerns between the bidirectional semantics, and the graphical user interface. The forward semantics translate the epicyclic structure into a simpler flat system of coordinates, which isolates the backwards semantics from the rendering engine. In other words, the bidirectional execution code is entirely agnostic to how curves are rendered on a screen and interact with user input.

We take a similar approach to handling user input: when a user manipulates a graphical curve, rather than directly update the AST, we produce a `SceneEdit` value which references a uniquely identified epicycle, along with the target change.

3.5.1 Forward Evaluation

We first present a general big-step semantics for our DSL – our evaluation has no recursion, state, or control flow, so small-step would not give any more insight.

We define our semantics in terms of a judgement $\sigma, t \vdash e : \tau \Downarrow v$, meaning under environment σ at time t, expression e with type τ evaluates to value v. The numeric part of our language does not depend on t, unlike the curve part, but is included for uniformity.

$$\frac{}{\sigma, t \vdash n : \mathbb{Z} \Downarrow n}\ \text{E-LitInt} \qquad \frac{}{\sigma, t \vdash n : \mathbb{Q} \Downarrow n}\ \text{E-LitRat} \qquad \frac{x : \tau \in \sigma}{\sigma, t \vdash x : \tau \Downarrow \sigma(x)}\ \text{E-Var}$$

$$\frac{\sigma, t \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma, t \vdash e_2 : \eta \Downarrow v_2}{\sigma, t \vdash e_1 + e_2 : \eta \Downarrow v_1 + v_2}\ \text{E-Add} \qquad \frac{\sigma, t \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma, t \vdash e_2 : \eta \Downarrow v_2}{\sigma, t \vdash e_1 \times e_2 : \eta \Downarrow v_1 \times v_2}\ \text{E-Mult}$$

$$\frac{\sigma, t \vdash e : \mathbb{Q} \Downarrow v}{\sigma, t \vdash e^{-1} : \mathbb{Q} \Downarrow \frac{1}{v}}\ \text{E-Recip} \qquad \frac{\sigma, t \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma[x \mapsto v_1], t \vdash o : \tau \Downarrow v}{\sigma, t \vdash \text{let } x = e_1 \text{ in } o : \tau \Downarrow v}\ \text{E-Let}$$

$$\frac{\sigma, t \vdash e_r : \mathbb{Q} \Downarrow v_r \qquad \sigma, t \vdash e_\omega : \mathbb{Z} \Downarrow v_\omega \qquad \sigma, t \vdash e_\phi : \mathbb{Q} \Downarrow v_\phi}{\sigma, t \vdash \text{epicycle}(e_r, e_\omega, e_\phi) : \text{Curve} \Downarrow v_r \cdot e^{i(v_\omega \cdot t + v_\phi)}}\ \text{C-Epicycle}$$

$$\frac{\sigma, t \vdash c_1 : \text{Curve} \Downarrow v_1 \qquad \sigma, t \vdash c_2 : \text{Curve} \Downarrow v_2}{\sigma, t \vdash c_1 \oplus c_2 : \text{Curve} \Downarrow v_1 + v_2}\ \text{C-Append}$$

3.5.2 Backward Evaluation Framework

One of the main goals of this project is to evaluate and compare multiple different approaches to bidirectional semantics. We will now introduce what this means in general, and then present the specific approaches we have implemented.

Just as we have a forward evaluation relation $\Downarrow$, we also have a backward evaluation relation $\Uparrow$. This relation is more complex than the forward relation, as it has to update both the expression and the environment. We define the general judgement as $\sigma, t \vdash e : \tau \Uparrow v' \rightsquigarrow (\sigma', e')$, meaning that for expression $e : \tau$ under environment σ at time t, to update the value to v', we can produce an updated environment σ' and an updated expression e'. In other words, this relation defines a 'solution' to e, σ for some v', such that $\sigma', t \vdash e' : \tau \Downarrow v'$.

This leads to a few properties we would like to hold for our semantics:

PutGet (correctness): If backwards evaluation produces an update, re-evaluating the result yields the target value:

$$\frac{\sigma, t \vdash e : \tau \Uparrow v' \rightsquigarrow (\sigma', e')}{\sigma', t \vdash e' : \tau \Downarrow v'} \textsc{PutGet}$$

GetPut (stability): Updating to the current value produces no change:

$$\frac{\sigma, t \vdash e : \tau \Downarrow v \qquad \sigma, t \vdash e : \tau \Uparrow v \rightsquigarrow (\sigma', e')}{\sigma' = \sigma \qquad e' = e} \textsc{GetPut}$$

PutPut (consistency): If we update to a value, then update again to a new value, this is the same as just updating to the new value:

$$\frac{\sigma, t \vdash e : \tau \Uparrow v' \rightsquigarrow (\sigma', e') \qquad \sigma', t \vdash e' : \tau \Uparrow v'' \rightsquigarrow (\sigma'', e'')}{\sigma, t \vdash e : \tau \Uparrow v'' \rightsquigarrow (\sigma'', e'')} \textsc{PutPut}$$

The **PutPut** property is a generalisation of a looser property of *convergence*: if we update to a value, then update again to the same value, this is the same as just updating to that value. This is essentially a stability property of the backwards semantics, and is very important for the user experience: if the user drags a curve to a new position, and then holds it there, the curve should not start 'drifting' around as the backwards semantics continue.

Defining the backwards semantics is the main technical challenge of this project, and the field of bidirectional programming as a whole. One challenge is the introduction of *nondeterminism*. In the forward direction, most semantics are deterministic – every expression will evaluate to a single, unique value.

In the context of our language, we observe that while the backwards judgement operates over the entire AST, there is a clear separation of concerns between the numeric and curve parts of the language. At the curve level, our numbering pass assigns each `epicycle` node a unique identifier and thus the backwards mapping is fully deterministic: within epicycle(e_r, e_ω, e_ϕ), an update to one of its parameters targets exactly one of the three sub-expressions. All non-determinism therefore is confined exclusively to the numeric part of the language. For an expression e with k parameter leaves (`Param` nodes), we define the candidate set $\mathcal{C}(e, \sigma, v')$ as the set of all environment updates σ' such that $\sigma', t \vdash e : \tau \Downarrow v'$. Then:

- **Literals** yield $|\mathcal{C}| = 0$ if $n \neq v'$, and $|\mathcal{C}| = 1$ if $n = v'$.
- **Parameters** yield $|\mathcal{C}| = 1$: set $\sigma'(x) = v'$
- **Recip** yields $|\mathcal{C}| = 1$: solve uniquely for e_1 as $v_1' = 1/v'$

The compound expressions are where nondeterminism appears:

- **Add** $e_1 + e_2 = v'$ admits the candidate set $\mathcal{C} = \{(v' - v_2, v_2) | v_2 \in \mathbb{R}\}$. We will resolve this infinite set in various ways in our different semantics, which constrain the search space to a finite set.
- **Mul** is similar, with the extra case of $v_1 = 0$ or $v_2 = 0$ requiring special treatment

In contrast, the `Let` construct introduces an entirely different kind of non-determinism: rather than algebraic ambiguity, we have *aliasing*. Consider

$$\text{let } r = 10 \text{ in } \text{epicycle}(r, 1, 0) \oplus \text{epicycle}(r, 3, 0)$$

A backward update targeting the first epicycle's radius to 12 produces two distinct update candidates: update the bound variable r to 12, propagating the change to both epicycles, or rewrite the AST to break the sharing, and introduce a new variable or constant multiplier to only update the first epicycle. Unlike the numeric cases, these two candidates are structurally and *semantically* distinct. The candidate set for `Let` is bounded by the number of usage sites of the bound variable, but the impact of each choice is global.

This characterisation is a direct consequence of domain restriction. In a general-purpose language, the ambiguity space is unbounded: arbitrary control flow, functions, and complex combinators make the candidate set infinite and unstructured. In our DSL, we are able to keep the candidate set finite in many cases, pushing the non-determinism to a manageable level.

3.6 Non-Deterministic Semantics

The first bidirectional semantics we investigate is a heuristic approach, proposed by Mayer et al. as an improvement over the original trace-based Sketch-n-Sketch semantics[2]. This defines an inherently non-deterministic backwards semantics, and defers the resolution of this to some external heuristic – in their case, asking the user to choose from a small set of candidates. In our case, we instead resolve the ambiguity by defining a heuristic function. As previously discussed, our DSL is significantly simpler than the *LittleLeo* language used in the paper, so we redefine their semantics in our context, noting that the majority of the complex cases are no longer present.

For consistency, we continue to use our judgement notation. Our judgement form $\sigma, t \vdash e : \tau \Downarrow v$ translates to $(E \vdash e) \Rightarrow v$, and $\sigma, t \vdash e : \tau \Uparrow v' \rightsquigarrow (\sigma', e')$ translates to $(E \vdash e) \Leftarrow v' \rightsquigarrow (E' \vdash e')$ by Mayer et al.. Some extra differences worth noting are that our environments are type-indexed, and we use De-Bruijn indices to avoid having to manually pattern match over environments in rules like U-Var[3]. For this semantics, the forward evaluation is the same as the standard semantics defined in section 3.5.1, but with our new backwards evaluation defined as follows:

$$\frac{}{\sigma \vdash n : \eta \Uparrow n' \rightsquigarrow (\sigma, n')}\ \text{U-Const} \qquad \frac{}{\sigma \vdash x : \tau \Uparrow v' \rightsquigarrow (\sigma[x \mapsto v'], x)}\ \text{U-Var}$$

$$\frac{\sigma \vdash e_2 : \eta \Downarrow v_2 \qquad \sigma \vdash e_1 : \eta \Uparrow v' - v_2 \rightsquigarrow (\sigma_1, e_1')}{\sigma \vdash e_1 + e_2 : \eta \Uparrow v' \rightsquigarrow (\sigma_1, e_1' + e_2)}\ \text{U-Add-1}$$

$$\frac{\sigma \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma \vdash e_2 : \eta \Uparrow v' - v_1 \rightsquigarrow (\sigma_2, e_2')}{\sigma \vdash e_1 + e_2 : \eta \Uparrow v' \rightsquigarrow (\sigma_2, e_1 + e_2')}\ \text{U-Add-2}$$

$$\frac{\sigma \vdash e_2 : \eta \Downarrow v_2 \qquad v_2 \neq 0 \qquad v'/v_2 \in \eta \qquad \sigma \vdash e_1 : \eta \Uparrow v'/v_2 \rightsquigarrow (\sigma_1, e_1')}{\sigma \vdash e_1 \times e_2 : \eta \Uparrow v' \rightsquigarrow (\sigma_1, e_1' \times e_2)}\ \text{U-Mul-1}$$

$$\frac{\sigma \vdash e_1 : \eta \Downarrow v_1 \qquad v_1 \neq 0 \qquad v'/v_1 \in \eta \qquad \sigma \vdash e_2 : \eta \Uparrow v'/v_1 \rightsquigarrow (\sigma_2, e_2')}{\sigma \vdash e_1 \times e_2 : \eta \Uparrow v' \rightsquigarrow (\sigma_2, e_1 \times e_2')}\ \text{U-Mul-2}$$

$$\frac{\sigma \vdash e : \mathbb{Q} \Uparrow 1/v' \rightsquigarrow (\sigma', e')}{\sigma \vdash e^{-1} : \mathbb{Q} \Uparrow v' \rightsquigarrow (\sigma', e'^{-1})}\ \text{U-Recip}$$

$$\frac{\sigma \vdash e_1 : \eta \Downarrow v_1 \qquad \sigma[x \mapsto v_1] \vdash o : \tau \Uparrow v' \rightsquigarrow (\sigma_{\text{outer}}[x \mapsto v_1'], o') \qquad \sigma_{\text{outer}} \vdash e_1 : \eta \Uparrow v_1' \rightsquigarrow (\sigma', e_1')}{\sigma \vdash \text{let } x = e_1 \text{ in } o : \tau \Uparrow v' \rightsquigarrow (\sigma', \text{let } x = e_1' \text{ in } o')}\ \text{U-Let}$$

We omit the trivial cases for `epicycle` and $\oplus$ for brevity.

As mentioned, these semantics are non-deterministic. The rules U-Add-1 and U-Add-2 both match for the same expression, but produce different updates. The same is true for U-Mul-1 and U-Mul-2. Due to our frequency domain being $\mathbb{Z}$, we also include checks that candidate updates are valid for this domain. This also has the added benefit of acting as a natural pruning mechanism, keeping the candidate small by filtering out invalid updates.

Aside from making edge cases more explicit, simplification, and adjustment to our specific language, these rules are virtually identical to the ones defined by Mayer et al., with the major exception of the U-Let rule. The original paper's semantics propagates the update through both variables independently, and then resolves conflicts separately through the use of an "environment merge operation" $E_1 \,{}^{e_1}\!\oplus^{e_2} E_2$, which reconciles the two candidate environments in the case they diverge. In contrast, we choose to evaluate the two sub-updates sequentially rather than non-deterministically, avoiding the need for this environment merge. This would violate the PutGet property, which we guard against by adding an extra check after construction, filtering out cases where PutGet would fail *specifically* because of sequential threading. This is practical because our candidate set is small and finite thanks to the domain restriction of our language; in the paper's general setting, this approach would be infeasible.

To resolve the non-determinism, we define a cost model that ranks candidates by how "invasive" the edit is to the original program. We define an **edit depth** ordering:

$$\text{NoEdit} < \text{LitEdit} < \text{LocalParam} < \text{OuterParam}(n) < \text{ImpossibleEdit}$$

[3]Variable names are used throughout judgements for clarity

with the following intuitive meaning:

- **NoEdit** means the candidate is the same as the original
- **LitEdit** means only a literal was changed
- **LocalParam** means a parameter in the innermost scope was changed
- **OuterParam**(n) means a parameter n scopes out was changed
- **ImpossibleEdit** means the edit depth could not be determined (e.g. when comparing two structurally different ASTs)

This ordering captures the intuition that edits closer to the point of use are preferred over edits that propagate through variable binding, essentially trying to make edits as 'local' as possible. We take this approach to try and make edits more predictable to the user – an edit with a lower edit depth is generally more desirable than one with a higher edit depth. This is an opinionated, but principled design decision we will evaluate later, based on the assumption that the user probably wants to change the thing they are clicking on *without* altering everything else. Our heuristic strategy then evaluates all possible backwards updates, and selects the candidate with the minimum edit depth.

We implement the non-determinism backwards rules using Haskell's `List` monad, which elegantly models non-deterministic computation: each rule returns a list of candidates, and the monadic bind operator composes these choices. The edit depth is computed by a structural comparison function `diffExpr` that traverses two ASTs in parallel, returning the most invasive edit encountered. This is where the `Typeable` constraint on `Let` is required – we can only compare two `Let` nodes if their binders are of the same type, otherwise we fall back to ImpossibleEdit. Importantly, when evaluating multiple branches such as for an `Add` node, we take the maximum edit depth from either branch, rather than the sum. This choice is justified by the branching nature of our update semantics. Each branch rule updates *either* operand independently, and never both simultaneously. Therefore, any single candidate updates up to one leaf node, and so we use the `max` combinator to propagate this cost up through the tree. This ranking is also intuitive – the 'danger' of an edit is determined by the most invasive change it makes, rather than the total number of changes; which would also remain useful if future extensions to the semantics permitted composite edits.

We define two main strategies with this algorithm as a base: a **naive update**, which simply applies the first candidate returned by the backwards semantics, and a **heuristic update**, which applies the minimum edit depth candidate.

3.7 Delta-Based Semantics

In section 2.3.1 we introduced the idea of an *edit lens*. As discussed, lenses struggle to handle the presence of shared bindings, however, the *delta-based* approach to edit lenses bears further investigation. Zhang et al. propose a new general purpose delta-based framework for bidirectional programming with direct manipulation. Similarly to edit lenses, this framework defines a *Delta Language* DM which describes edits to the program. Unlike the simpler framework used by Mayer et al., this framework allows for *fusion*, i.e. the synthesis of program modifications to resolve edits in the case of ambiguity. When a user performs a direct manipulation, this gets translated to a delta in DM, which is then applied to the program to produce an updated program. This extra layer of indirection separates the concerns of the user input and the program update more clearly. We implement this framework as a second bidirectional semantics for our DSL for comparison to the non-deterministic semantics.

We begin by defining our delta language in fig. 3.4. We have two kinds of deltas for the numeric and curve parts of our language.

The numeric deltas allow either direct replacement, or incremental updates. Furthermore, our numeric delta language forms a monoid under $\circ$, with identity id_n. On the other hand, the curve deltas are more structured, serving as an 'entrypoint' to apply numeric deltas to specific sub-expressions of the curve.

This delta language, while significantly simplified from the paper's original version, is still more expressive than our implementation requires. Notably, the UI of our application only supports additive deltas $+_\Delta n$, despite the framework supporting more complex edits. Similarly, the $\oplus_\Delta$ operator is not used, but would allow simultaneously editing multiple epicycles at once. We present the full language for completeness, noting that adding the extra UI functionality to support the full language would be a straightforward extension for future work.

NumDelta	dv_n	::=	id_n	identity
		|	$\mathrm{repl}_n\ n$	replacement
		|	$+_\Delta n$	addition
		|	$\times_\Delta n$	multiplication
		|	$dv_{n1} \circ_\Delta dv_{n2}$	composition
CurveDelta	dv_c	::=	id_c	identity
		|	$\mathrm{epicycle}_\Delta(id_c, dv_{nr}, dv_{n\omega}, dv_{n\phi})$	epicycle update
		|	$dv_{c1} \oplus_\Delta dv_{c2}$	multi update
CurveId	id_c	::=	c_1	
		|	c_2	
		|	$\ldots$	
		|	c_k	

Figure 3.4: Delta language for our DSL

$$\frac{}{id_n \triangleleft n \rightsquigarrow n}\ \text{D-Id-N} \qquad \frac{}{repl_n n' \triangleleft n \rightsquigarrow n'}\ \text{D-Repl-N} \qquad \frac{}{+_\Delta n' \triangleleft n \rightsquigarrow n + n'}\ \text{D-Add-N}$$

$$\frac{}{\times_\Delta n' \triangleleft n \rightsquigarrow n \times n'}\ \text{D-Mul-N} \qquad \frac{dv_1 \triangleleft n \rightsquigarrow n' \qquad dv_2 \triangleleft n' \rightsquigarrow n''}{dv_2 \circ_\Delta dv_1 \triangleleft n \rightsquigarrow n''}\ \text{D-Comp-N}$$

Figure 3.5: Delta semantics for the numeric part of the language, adapted from [19]

We present the application semantics for our delta language in fig. 3.5 as the judgement $dv \triangleleft v \rightsquigarrow v'$, meaning applying delta dv to value v produces the updated value v'. Only the numeric deltas have a meaningful application semantics, as the curve deltas are instead used as structural directives for the backwards evaluation, to determine where in the AST to apply the numeric deltas. Note that $\circ_\Delta$ is *right-associative* semantically, meaning that $dv_1 \circ_\Delta dv_2$ applies dv_2 first, then dv_1.

We can now define the backwards semantics for our delta-based approach. We define a delta environment Δ as a separate mapping from variables to 'pending' deltas on their values, maintained alongside the value environment σ with the invariant that they share the same structure (i.e. the same variables are bound in both). The initial delta environment maps every variable to id_n. This is a slight deviation from the paper's combined representation v^{dv}, but is more natural for a statically typed implementation where values and deltas have different types. This separation however introduces a subtle complication. In the original combined environment, evaluating a variable inherently yields its *updated* value, as the delta and value are inseparable. By decoupling them, evaluation cannot solely depend on the value environment σ which would produce 'stale' data, violating the PutPut property. To resolve this, we define a slightly modified forward judgement $(\Delta \triangleright \sigma), t \vdash e : \tau \Downarrow v'$, representing the evaluated expression *after* applying all pending deltas in Δ to σ, replicating this original functionality. Consequently, backwards rules that rely on forward-evaluated values (such as D-Add-Mul and D-Add-Recip) must use this updated judgement.

We use the judgement: $\Delta \vdash dv \triangleleft e \rightsquigarrow (\Delta', e')$ reads as "under delta environment Δ, applying delta dv to expression e produces updated environment Δ' and updated expression e'". We keep σ available, but implicit unless explicitly used. The rules are described in fig. 3.6 for numeric deltas and fig. 3.7 for curve deltas.

Notation. The semantics make use of some new notation and functions we will define here:

- $\Delta_\mathbf{0}$ denotes the identity delta environment, i.e. $\forall x. \Delta_\mathbf{0}(x) = id_n$
- The *comparing operator* $\Delta_1 {}^{e_1}\!\times^{e_2} \Delta_2$ compares environments Δ_1 and Δ_2, and splits them into three parts: the common prefix Δ^M, the unique suffix of Δ_1, denoted as Δ_1^I, and the unique suffix of Δ_2, denoted as Δ_2^I. We define this in terms of a helper function $\mathrm{compare}(dv_1, dv_2, x, e_1, e_2)$ defined as

follows:

$$\text{compare}(dv_1, dv_2, x, e_1, e_2) = \begin{cases} (dv_1, id_n, id_n) & \text{if } dv_1 = dv_2 \text{ or } x \notin \text{fv}(e_2) \\ (dv_2, id_n, id_n) & \text{if } x \notin \text{fv}(e_1) \\ (dv^s, dv_1^p, dv_2^p) & \text{otherwise, where} (dv^s, dv_1^p, dv_2^p) = dv_1 \wedge_{\text{suffix}} dv_2 \end{cases}$$

${\Delta_1}^{e_1} \times^{e_2} \Delta_2$ is therefore the pointwise application of compare, producing $(\Delta^M, \Delta_1^I, \Delta_2^I)$.

- $dv_1 \wedge_{\text{suffix}} dv_2$ is the *longest common suffix* of dv_1 and dv_2
- $\text{resolve}(\Delta_1, \Delta_2, \Delta, e_1, e_2) = (\Delta^M, e_1 \odot \Delta_1^I, e_2 \odot \Delta_2^I)$ where $(\Delta^M, \Delta_1^I, \Delta_2^I) = {\Delta_1}^{e_1} \times^{e_2} \Delta_2$
- $e \odot \Delta$ is the *embedding* operator, which fuses the delta environment Δ into expression e by rewriting e according to the semantics of the deltas in Δ. In other words, given $e \odot \Delta = e'$ and $\sigma \vdash e \Downarrow v$, it should hold that $\sigma \vdash e' \Downarrow v'$ where $dv \triangleleft v \rightsquigarrow v'$. Specifically, this replaces each variable x in e with the transformation $\exp(\Delta(x), x)$, which transforms a delta value into an expression. We define this piecewise as:

$$\exp(dv, x) = \begin{cases} x & \text{if } dv = id_n \\ n + x & \text{if } dv = +_\Delta n \\ n \times x & \text{if } dv = \times_\Delta n \\ n' & \text{if } dv = repl_n n' \\ \exp(dv_1, \exp(dv_2, x)) & \text{if } dv = dv_1 \circ_\Delta dv_2 \end{cases}$$

  For example, $x * y \odot \{x \mapsto +_\Delta 5\} = (x + 5) * y$. Our implementation of the embedding operator differs slightly to the original. Whereas Zhang et al. rewrite deltas to lambda expressions (e.g. $+_\Delta n \mapsto \lambda x.x + n$), and then replace variables with applications of these lambdas, we instead directly rewrite at the use site. This is simpler, and more efficient for our purposes, but mentioned to note that these semantics could be extended to a significantly more complex language.

- Conflict resolution is encapsulated into the *environment merge operator* $\bowtie$[4]. Given two branches of backwards evaluation, it extracts the common environment Δ^M and embeds the remaining inconsistent parts:

$$\langle \Delta_1, e_1 \rangle \bowtie \langle \Delta_2, e_2 \rangle = (\Delta^M, e_1 \odot \Delta_1^I, e_2 \odot \Delta_2^I)$$

  where $(\Delta^M, \Delta_1^I, \Delta_2^I) = {\Delta_1}^{e_1} \times^{e_2} \Delta_2$ as defined above.

Variable Bindings: Unlike in the heuristic semantics, variable bindings *compose* deltas, rather than overwriting the environment with the new desired value. When we apply a new delta dv_1 to a variable which has a pending delta dv, we apply the composition $dv_1 \circ_\Delta dv$ to the variable.

Addition: We resolve the non-determinism of addition by propagating the delta to only one side of the addition, and leaving the other side unchanged. This is a somewhat arbitrary decision, but matches the decision made by Zhang et al. in their semantics. We apply an addition delta differently depending on the structure of the expression: if the addition is at the top level, we simply propagate the delta to the left operand (D-Add-Add). If the expression is a multiplication, we divide the delta by the value of the right operand, and then propagate it to the left operand (D-Add-Mul). Mathematically, if $e_1 \times v_2$ needs to be changed by d, then e_1 needs to be changed by d/v_2. The same intuition applies to the reciprocal case, which uses the identity $1/(1/v + d) - v = (v + d)^{-1}$ (D-Add-Recip). The rules for multiplication follow the same pattern.

Due to the frequency domain being restricted to $\eta = \mathbb{Z}$, we have to split the D-Add-Mul rule into two cases, as division has now become a partial function. We add the condition to D-Add-Mul that $d/v_2 \in \eta$, and if this fails, we fall back to the new rule D-Add-Embed, which embeds the delta directly into the expression as an addition.

[4] In [19], this is defined as ${\Delta_1}^{e_1} \oplus^{e_2} \Delta_2$, which causes ambiguity with our curve composition operator $c_1 \oplus c_2$

Let Bindings: Unlike the non-deterministic semantics, we evaluate the two branches in parallel. First, the incoming delta dv is propagated through the o to extract dv_x, the delta requested for the bound variable x. Then, dv_x is propagated to the bound value e_1 to extract the updated value e_1'. Note that both propagations are performed under the original delta environment Δ. This produces two distinct candidate environments Δ_{val} and Δ_{body}, which are then merged together using the environment merge operator $\bowtie$ to produce the final updated environment Δ^M and expression let $x = e_1''$ in o''. As the environments are of different sizes, we use a slightly modified version of $\bowtie$ defined as $\bowtie_{\text{let}}$. This accounts for o' being in an extended lexical scope (having access to x) compared to e_1', and so we shift its free variable checks by 1 De-Bruijn index, to prevent the algorithm from incorrectly comparing separate variables (and, as an implementation detail, to ensure the types line up correctly).

Curves: Similarly, we evaluate the updates to all 3 parameters of an epicycle in parallel, and resolve any conflicts in the environment. This avoids subtle cases of PutPut failure when multiple parameters of an epicycle share a variable.

3.7.1 Structural Invariants

The application of the embedding operator $e \odot \Delta$ resolves shared variable conflicts by replacing shared references with independent sub-expressions with the deltas applied. This causes the AST to grow, however, we note that under the update semantics, this reaches a fixed point. At a high level, this fixed point is when all variables are surrounded by literal addition and/or multiplication nodes on their left, so rules D-Add-Add and D-Mul-Mul do not produce any conflicts in the environment, and thus do not trigger the embedding operator.

Theorem 2 (Fixed Point of Delta Updates)**.** *Given a finite initial AST e_0 and an infinite sequence of arbitrary edits $E = (\varepsilon_1, \varepsilon_2, \dots)$, the shape of the AST converges to a fixed point where the embedding operator $\odot$ is no longer triggered by variable conflicts.*

Proof. The embedding operator only expands the AST when the environment merge operator ($\bowtie$) detects a conflict. Specifically, when a shared variable x receives divergent deltas from two different branches of an update.

When this conflict is resolved by embedding, the variable x is replaced at the use-site with $\exp(dv, x)$, which wraps x in literal arithmetic nodes (e.g. transforming x into $n + x$ or $n \times x$). We call this *literal shielding*, as it effectively shields the variable from being updated directly in future, preventing conflicts.

Consider a subsequent edit that propagates down this newly shielded branch. According to our additive and multiplicative propagation rules (D-Add-Add and D-Mul-Mul), incoming deltas are routed to the left operand (e_1). Because this left operand is now a literal n, the D-Lit rule triggers, which evaluates the delta and updates the literal in place, leaving Δ unchanged. This means that the right operand e_2's delta is also unchanged, remaining as the default of id_n.

Once all usage sites of a shared variable x have been shielded by at least one conflict resolution, any subsequent edit to x will be fully absorbed by the literal shields. Consequently, the delta environment's value for x will be strictly id_n in all branches. When $\bowtie$ compares these environments, it evaluates $id_n = id_n$, detects no conflict, and passes the identity environment upward without invoking $\odot$.

Because the AST is of finite size, it contains a finite number of shared variables, and thus there is a finite number of possible conflicts that can occur before every variable reference is shielded. Therefore, given an infinite amount of edits, we always reach a fixed point where the embedding operator is never triggered again. □

3.8 Comparison of Strategies

To illustrate the differences between the two strategies, we present a high level worked example. In the interest of brevity, we do this somewhat informally, with the derivations elided.

Consider the example program $p = \text{let } r = 10 \text{ in epicycle}(r, 1, 0) \oplus \text{epicycle}(r, 3, 0)$, with the user dragging a point on the first epicycle to change its radius towards 15.[5]

Under the heuristic semantics, backward judgment receives the target value 15 for the radius parameter of the first epicycle. The rules navigate to epicycle$(r, 1, 0)$, extract r, and enumerate candidates for making

[5] In our real UI, instantaneous updates are impossible, so the first update would be a smaller, incremental one such as 10.05

Base Cases

$$\frac{}{\Delta \vdash id_n \triangleleft e \rightsquigarrow (\Delta, e)}\ \text{D-Id} \qquad \frac{dv \triangleleft n \rightsquigarrow n'}{\Delta \vdash dv \triangleleft n \rightsquigarrow (\Delta, n')}\ \text{D-Lit}$$

$$\frac{\Delta(x) = dv}{\Delta \vdash dv_1 \triangleleft x \rightsquigarrow (\Delta[x \mapsto dv_1 \circ_\Delta dv], x)}\ \text{D-Var} \qquad \frac{}{\Delta \vdash repl_n n' \triangleleft e \rightsquigarrow (\Delta_{\mathbf{0}}, n')}\ \text{D-Repl}$$

Additive Propagation

$$\frac{\Delta \vdash +_\Delta d \triangleleft e_1 \rightsquigarrow (\Delta_1, e_1') \qquad \langle \Delta_1, e_1' \rangle \bowtie \langle \Delta, e_2 \rangle \rightsquigarrow (\Delta^M, e_1'', e_2')}{\Delta \vdash +_\Delta d \triangleleft e_1 + e_2 \rightsquigarrow (\Delta^M, e_1'' + e_2')}\ \text{D-Add-Add}$$

$$\frac{d/v_2 \in \eta \qquad \Delta \vdash +_\Delta (d/v_2) \triangleleft e_1 \rightsquigarrow (\Delta_1, e_1') \qquad \begin{array}{c}(\Delta \triangleright \sigma) \vdash e_2 \Downarrow v_2 \\ \langle \Delta_1, e_1' \rangle \bowtie \langle \Delta, e_2 \rangle \rightsquigarrow (\Delta^M, e_1'', e_2')\end{array}}{\Delta \vdash +_\Delta d \triangleleft e_1 \times e_2 \rightsquigarrow (\Delta^M, e_1'' \times e_2')}\ \text{D-Add-Mul}$$

$$\frac{(\Delta \triangleright \sigma) \vdash e_2 \Downarrow v_2 \qquad d/v_2 \notin \eta}{\Delta \vdash +_\Delta d \triangleleft e_1 \times e_2 \rightsquigarrow (\Delta^M, d + (e_1 \times e_2))}\ \text{D-Add-Embed}$$

$$\frac{(\Delta \triangleright \sigma) \vdash e \Downarrow v \qquad d_x = 1/(1/v + d) - v \qquad \Delta \vdash +_\Delta d_x \triangleleft e \rightsquigarrow (\Delta', e')}{\Delta \vdash +_\Delta d \triangleleft e^{-1} \rightsquigarrow (\Delta', e'^{-1})}\ \text{D-Add-Recip}$$

$$\frac{(\Delta \triangleright \sigma) \vdash e \Downarrow v \qquad v_1/v + d = 0}{\Delta \vdash +_\Delta d \triangleleft e^{-1} \rightsquigarrow (\Delta, d + e^{-1})}\ \text{D-Add-Recip-Embed}$$

Multiplicative Propagation

$$\frac{\Delta \vdash \times_\Delta m \triangleleft e_1 \rightsquigarrow (\Delta_1, e_1') \qquad \Delta_1 \vdash \times_\Delta m \triangleleft e_2 \rightsquigarrow (\Delta_2, e_2')}{\Delta \vdash \times_\Delta m \triangleleft e_1 + e_2 \rightsquigarrow (\Delta_2, e_1' + e_2')}\ \text{D-Mul-Add}$$

$$\frac{\Delta \vdash \times_\Delta m \triangleleft e_1 \rightsquigarrow (\Delta_1, e_1') \qquad \langle \Delta_1, e_1' \rangle \bowtie \langle \Delta, e_2 \rangle \rightsquigarrow (\Delta^M, e_1'', e_2')}{\Delta \vdash \times_\Delta m \triangleleft e_1 \times e_2 \rightsquigarrow (\Delta^M, e_1'' \times e_2')}\ \text{D-Mul-Mul}$$

$$\frac{\Delta \vdash \times_\Delta (1/m) \triangleleft e \rightsquigarrow (\Delta', e')}{\Delta \vdash \times_\Delta m \triangleleft e^{-1} \rightsquigarrow (\Delta', e'^{-1})}\ \text{D-Mul-Recip} \qquad \frac{m = 0}{\Delta \vdash \times_\Delta m \triangleleft e^{-1} \rightsquigarrow (\Delta, m \times e^{-1})}\ \text{D-Mul-Recip-Embed}$$

Composition

$$\frac{\Delta \vdash dv_2 \triangleleft e \rightsquigarrow (\Delta', e') \qquad \Delta' \vdash dv_1 \triangleleft e' \rightsquigarrow (\Delta'', e'')}{\Delta \vdash dv_1 \circ_\Delta dv_2 \triangleleft e \rightsquigarrow (\Delta'', e'')}\ \text{D-Comp}$$

Let

$$\frac{\begin{array}{c}\sigma \vdash e_1 \Downarrow v_1 \qquad \sigma[x \mapsto v_1], \Delta[x \mapsto id_n] \vdash dv \triangleleft o \rightsquigarrow (\Delta_{body}[x \mapsto dv_x], o') \\ \Delta \vdash dv_x \triangleleft e_1 \rightsquigarrow (\Delta_{val}, e_1') \qquad \langle \Delta_{val}, e_1' \rangle \bowtie_{\text{let}} \langle \Delta_{body}, o' \rangle \rightsquigarrow (\Delta^M, e_1'', o'')\end{array}}{\Delta \vdash dv \triangleleft \text{let } x = e_1 \text{ in } o \rightsquigarrow (\Delta^M, \text{let } x = e_1'' \text{ in } o'')}\ \text{D-Let}$$

Figure 3.6: Backwards evaluation rules for the numeric part of our delta language, adapted from [19].

Curves

$$\frac{\Delta \vdash dv_r \triangleleft e_r \rightsquigarrow (\Delta_r, e'_r) \qquad \Delta \vdash dv_\omega \triangleleft e_\omega \rightsquigarrow (\Delta_\omega, e'_\omega) \qquad \Delta \vdash dv_\phi \triangleleft e_\phi \rightsquigarrow (\Delta_\phi, e'_\phi) \qquad \langle \Delta_r, e'_r \rangle \bowtie \langle \Delta_\omega, e'_\omega \rangle \bowtie \langle \Delta_\phi, e'_\phi \rangle \rightsquigarrow (\Delta^M, e''_r, e''_\omega, e''_\phi)}{\Delta \vdash \text{epi}_\Delta(k, dv_r, dv_\omega, dv_\phi) \triangleleft \text{epi}(k, e_r, e_\omega, e_\phi) \rightsquigarrow (\Delta^M, \text{epi}(k, e''_r, e''_\omega, e''_\phi))} \text{ D-Epi-Match}$$

$$\frac{k \neq k'}{\Delta \vdash \text{epi}_\Delta(k, dv_r, dv_\omega, dv_\phi) \triangleleft \text{epi}(k', e_r, e_\omega, e_\phi) \rightsquigarrow (\Delta, \text{epi}(k', e_r, e_\omega, e_\phi))} \text{ D-Epi-Skip}$$

$$\frac{\Delta \vdash dv_c \triangleleft c_1 \rightsquigarrow (\Delta_1, c'_1) \qquad \Delta \vdash dv_c \triangleleft c_2 \rightsquigarrow (\Delta_2, c'_2) \qquad \langle \Delta_1, c'_1 \rangle \bowtie \langle \Delta_2, c'_2 \rangle \rightsquigarrow (\Delta^M, c''_1, c''_2)}{\Delta \vdash dv_c \triangleleft c_1 \oplus c_2 \rightsquigarrow (\Delta^M, c''_1 \oplus c''_2)} \text{ D-Append-Target}$$

$$\frac{\Delta \vdash dv_L \triangleleft c_1 \rightsquigarrow (\Delta_1, c'_1) \qquad \Delta_1 \vdash dv_R \triangleleft c_2 \rightsquigarrow (\Delta_2, c'_2)}{\Delta \vdash dv_L \oplus_\Delta dv_R \triangleleft c_1 \oplus c_2 \rightsquigarrow (\Delta_2, c'_1 \oplus c'_2)} \text{ D-Append-Split}$$

Figure 3.7: Backwards evaluation rules for the curve part of our delta language, adapted from [19]. For brevity, we abbreviate *epicycle* as *epi*

r evaluate to 15. The only candidate to update is r itself, since it is a parameter and not a compound expression. The updated program therefore becomes $p' = \text{let } r = 15 \text{ in epicycle}(r, 1, 0) \oplus \text{epicycle}(r, 3, 0)$: both epicycles are affected, and the second epicycle's radius is also changed to 15 despite the user only interacting with the first.

In contrast, the delta semantics synthesise the edit into an additive delta $\text{epicycle}_\Delta(1, +_\Delta 5, id_n, id_n)$. This propagates through the $\oplus$ (D-Append-Target), evaluating both epicycles with the same delta environment. The left branch ($\text{epicycle}(r, 1, 0)$) matches the target epicycle (D-Epi-Match) and propagates $+_\Delta 5$ to r, composing the delta into the environment. The right branch ($\text{epicycle}(r, 3, 0)$) does not match the target epicycle (D-Epi-Skip) and so remains unchanged. The merge operator $\bowtie$ then sees that r carries $+_\Delta 5$ in the left delta environment, but id_n in the right delta environment, and r is free in both expressions. This is a conflict, and so the longest common suffix is extracted (in this case, there is none), and the residual deltas ($\{+_\Delta 5\}$ and $\{id_n\}$ respectively) are embedded into both branches, synthesising the new program $p' = \text{let } r = 10 \text{ in epicycle}(5 + r, 1, 0) \oplus \text{epicycle}(r, 3, 0)$. Unlike the heuristic semantics, we have synthesised a new program which only updates the target epicycle, leaving the second epicycle unchanged.

This example demonstrates the key difference between the two strategies: the heuristic semantics tries to make the minimal edit to the original AST's *structure*, resulting in a more invasive edit to the program, whereas the delta semantics tries to make the minimal edit to the program, resulting in a more invasive edit to the AST. Neither approach is universally 'correct', the preferred behaviour depends entirely on the user's intention, which cannot be perfectly inferred. We evaluate this tradeoff in more detail in chapter 4.

3.9 Compilation and Evaluation Pipeline

While not the focus of this project, we discuss the engineering and UI of the DSL's system in some more depth here.

The evaluation of our DSL follows a fairly simple pipeline:

1. The user writes a program in the HOAS surface syntax. The embedding-by-unembedding library converts this to a De-Bruijn indexed first order AST `AST Parsed`
2. `numberCurve` traverses the AST, assigning each `Epicycle` node a unique `EpicycleId`, producing `AST Numbered`
3. We wrap the numbered AST in a `CurveState` which pairs the AST and the current environment σ
4. The `evalScene` function evaluates the AST into a visual scene of evaluated curves, which is rendered to the `Canvas`

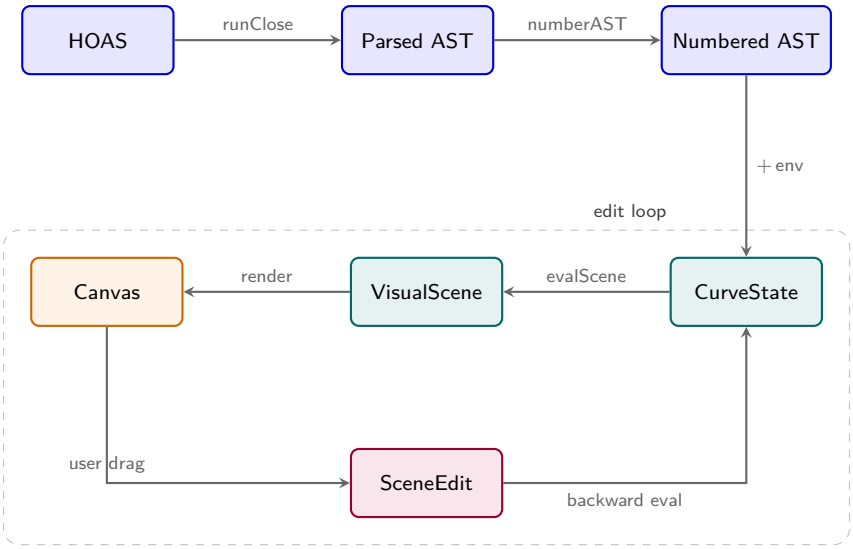


Figure 3.8: Compilation and evaluation pipeline. The upper path is the one-shot compilation from HOAS to a numbered AST. The lower loop runs every frame: `evalScene` evaluates the AST into a visual scene, which is rendered to the `Canvas`; user interactions produce `SceneEdit`s which are propagated back through the AST via the chosen backward strategy, producing an updated `CurveState`.

5. User interactions with the `Canvas` produce `SceneEdit`s, which are passed to the backwards semantics to produce an updated `CurveState`, which is then fed back into `evalScene` to update the rendered scene. This loop continues until the user is satisfied with the result and terminates the program.

This is demonstrated visually in fig. 3.8.

Chapter 4

Critical Evaluation

We evaluate our DSL by a few metrics. Firstly, we prove that the backwards semantics are correct with respect to some sensible laws – or justify their violation of the laws. Then, we compare the two strategies by evaluating their behaviour on a set of example programs and edits.

4.1 Formal Properties of the Backwards Semantics

We restate the main correctness laws for bidirectional semantics under a general semantic-agnostic notation: For a curve state S with AST e and environment σ, and a scene edit ϵ, we expect:

- **GetPut**: $\text{backwards}(S, \epsilon_{\text{id}}) = S$ – identity edit produces no change
- **PutGet**: $\text{backwards}(S, \epsilon) = S' \implies \text{eval}(S') = \text{apply}(\text{eval}(S), \epsilon)$ – applying the edit to the evaluated scene should produce the same result as evaluating the updated state
- **PutPut**: $\text{backwards}(\text{backwards}(S, \epsilon_1), \epsilon_2) = \text{backwards}(S, \epsilon_1 \circ \epsilon_2)$

4.2 Correctness of the non-deterministic semantics

We set out to mechanically prove these properties with the Lean theorem prover. The proofs are attached in the appendix, but will be discussed here. Note that we omit proofs for the curve part of the language and instead focus on the numeric part – the curve part (for these semantics) is a straightforward extension that does not introduce any new complications to the proofs.

The main observation is that for the non-deterministic semantics, many of the properties do *not* hold without certain extra pre-conditions.

4.2.1 GetPut (Stability)

This property states that an identity edit results in no changes to the source code.

We prove a general case of GetPut by induction:

```
theorem exprUpdate_getPut {n : Nat} (env : Env n) (e : Expr n) :
    ∀ r ∈ exprUpdate env e (evalExpr env e), updateResult_eq r e env
```

This theorem holds, but can be strengthened. Notably, it does not guarantee that the edit *exists*. We improve this by proving an extra property of *liveness* under GetPut – which is that updating with the identity edit always exists. Without this property, GetPut could vacuously hold in many cases if an edit didn't exist.

This exposes one *deliberate* edge case in our semantics, with regards to division by zero. Our forward semantics have no checks for division by zero, causing the normal IEEE 754 semantics of `Infinity` in these cases. We regard this as an acceptable compromise from a design perspective – the DSL is made for programmers, who wouldn't be surprised by this behaviour. However, for the backward semantics, this creates undefined behaviour of trying to invert division by zero. As such, our implementation of U-Recip explicitly guards against this, causing liveness to fail under the presence of 0^{-1} expressions. This is the only case where our backwards semantics are "partial". Therefore, to prove this, we define a theorem

`isValid e` which checks that e does not contain any 0^{-1} sub-expressions, and prove that liveness holds for all valid expressions:

```
theorem exprUpdate_getPut_liveness {n : Nat} (env : Env n)
    (e : Expr n) (h_valid : isValid env e) :
    ∃ r, r ∈ exprUpdate env e (evalExpr env e)
```

4.2.2 PutGet (Correctness)

This rule requires that the updated state causes the expression to evaluate to the target value. We observe that for these semantics, this rule does **not** hold in the general case, due to sharing. We demonstrated this with a counter-example in section 3.8, and argue that in some cases, this is the desired behaviour. PutGet fails under the presence of shared variables, because an update to a single epicycle may also update others. However, when some variable is shared, this is often the user's intention – the programmer is explicitly declaring a constraint between the two expressions. Thus, we consider this an acceptable violation. However, we can still prove PutGet under the precondition that an expression is "independent" – that is, it does not contain any shared variables.

```
theorem exprUpdate_putGet {n : Nat} (env : Env n) (e : Expr n) (targetVal : Rat):
  isIndependent e -> ∀ r ∈ exprUpdate env e targetVal,
    evalExpr r.env r.expr = targetVal
```

We note that the source of inspiration for these semantics, the Sketch-n-Sketch system of Mayer et al. also encounters this problem. As mentioned, they handle let-bindings in parallel and resolve them by merging the differing environments, with either a conservative merge, or an 'optimistic' merge. The conservative merge simply fails in the case of conflicts, requiring a similar restriction on the correctness property. The optimistic merge on the other hand, intentionally violates the correctness property in order to produce a better UX.

4.2.3 PutPut (Composability)

This dictates that applying two sequential updates is equivalent to applying only the final update. We observe that this property also does not hold in the general case, due to two factors. The failure of PutGet also causes PutPut to fail, since if the first update doesn't produce the target value, then the second update is not being applied to the correct state. Additionally, even without the presence of shared variables, we must weaken this statement due to the non-determinism of the semantics – we cannot say that PutPut holds for all possible update paths. For a simple example, consider $\sigma \vdash x + y \Downarrow 15$ for $\sigma(x) = 10, \sigma(y) = 5$. If we apply an edit to reach target value 20, there are two possible update paths: either x is updated to 15 or y is updated to 10. If we then apply another edit to reach target value 30, we add 10 to either x or y, producing the following possible final states: $\sigma_1 = \{x \mapsto 25, y \mapsto 5\}, \sigma_2 = \{x \mapsto 15, y \mapsto 15\}, \sigma_3 = \{x \mapsto 20, y \mapsto 10\}, \sigma_4 = \{x \mapsto 10, y \mapsto 20\}$. In contrast, updating directly to 30 produces only two possible final states: $\sigma_5 = \{x \mapsto 25, y \mapsto 5\}, \sigma_6 = \{x \mapsto 15, y \mapsto 15\}$. Thus, PutPut clearly does not hold for all paths.

We note that these semantics obey a very weakened version of PutPut which relies on 3 preconditions:

- The expression being updated is independent
- The expression is valid – i.e., no 0^{-1} sub-expressions
- *Deterministic Stability*, i.e., the intermediate updated value v_1 and the final updated value v_2 must be reachable via the same semantic path.

We will elaborate this third new condition by motivating its existence with an example. While it is tempting to suggest that PutPut will always hold for *some* update path, but not all, this is not true. Consider $\sigma = \{x \mapsto 0.1, y \mapsto 0.05\}$ and $e = x^{-1} + y^{-1}$. We initially have $\sigma, e \Downarrow 30$. If we apply an edit to reach target value 10, we solve either $x^{-1} = -10$, yielding $x = -0.1$, or $y^{-1} = 0$, which is *not* valid. Therefore we only have one candidate environment $\sigma' = \{x \mapsto -0.1, y \mapsto 0.05\}$. In contrast, if we first update to intermediate value 20, the x update is invalid, and we update y to produce $\sigma'' = \{x \mapsto 0.1, y \mapsto 0.1\}$. We then attempt to update σ'' to the same final target value 10, which proves to be impossible – either $y^{-1} = 0$ or $x^{-1} = 0$, both of which are invalid. Thus, PutPut does not hold for any update path.

The deterministic stability condition rules out this case, by requiring the same path for both the intermediate and final updates. More formally, this condition says that for v_1, v_2, the derivations of $\sigma \vdash e \Uparrow v_1 \rightsquigarrow (\sigma', e')$ and $\sigma' \vdash e' \Uparrow v_2 \rightsquigarrow (\sigma'', e'')$ must be structurally isomorphic. The previous example does not satisfy the condition because the intermediate update ($v_1 = 20$) takes the path of updating y, while the final update ($v_2 = 10$) takes the path of updating x, thus being excluded from our proof of PutPut.

Given the extensive restrictiveness of these preconditions, and the inherent complexity of formalising them in a theorem prover, we leave the mechanical verification of PutPut for these semantics as an open problem for future work. Instead, we evaluate this property empirically in section 4.6 by testing a large number of random updates and checking for violations.

It is worth nothing that for most expressions, this problem does not occur, as expressions usually contain some literals that can be modified to produce a valid update for both paths, which the solver prefers over mutating parameters.

4.3 Correctness of the Delta Semantics

These semantics are significantly more complex to mechanically reason about. There are multiple moving parts which complicate proofs significantly – for example, unlike the previous semantics, the AST does not strictly shrink over induction due to the presence of the embedding operator.

In the interest of time and given the complexity of these semantics, we attempt an LLM-assisted mechanisation of the proofs, which we argue can be verified rigorously – Lean's type checker is trusted enough to be able to confidently say that if the LLM's solution compiles, it works (so long as it does not contain shortcuts like `sorry`, `partial`, or `panic!` which are trivial to verify by inspection).

This required some extensive supervision and guidance. We wrote a simple Lean port of the main Delta Semantics types, and provided the model with the source of our Haskell semantics, along with the formal semantics written in this paper with the simple instruction of "port this to Lean". The initial response had an issue with proving that the backwards update terminated – it had written a naive mutually recursive pair of functions which did not obviously decrease. Further prompting led the model to suggest a 'fuel'-based solution, where a natural number of 'fuel' that decreases every inductive step is used to guarantee termination: by picking a reasonably high amount of fuel (e.g. based on the size of the expression), we can ensure termination in all normal cases while not having to prove it based on the structure of the induction.

We found this an unsatisfying solution, and pushed for a more rigorous analysis. We were able to produce more rigorous solution by informing the model of two important properties of the language and semantics: the *embedding convergence* property discussed in theorem 2, and the *normal form* of our AST discussed in section 3.4.2. These allowed writing a proof that did not depend on mutual recursion and thus could provably terminate.

Specifically, we partition the proofs into three layers over the sub-structures of the AST: *Numeric*, *Curve*, and *Program* (let bindings which finally evaluate to a curve). Some proofs are split into multiple sub-theorems which handle individual cases individually.

GetPut is straightforward for these semantics: the identity delta should leave the environment and expression unchanged:

```
theorem backwardEvalProgram_id {n : Nat} (vals : Env n)
    (deltas : DeltaEnv n) (P : Program n) :
    backwardEvalProgram .id vals deltas P = (deltas, P)
```

We prove PutGet via standard induction. Specifically, we prove that applying a delta and re-evaluating yields exactly the delta applied to the original evaluation, i.e.

```
theorem backwardEvalAtomic_putGet {n : Nat} (vals : Env n) :
    ∀ (e : NumExpr n) (d : AtomicNumDelta) (deltas : DeltaEnv n),
      evalNumExpr ((backwardEvalAtomic d vals deltas e).1.apply vals)
                  (backwardEvalAtomic d vals deltas e).2 =
        d.apply (evalNumExpr (deltas.apply vals) e)
```

This is more involved than GetPut and requires more in-depth induction. Furthermore, we define some lemmas to simplify this proof, some of which we have already discussed. These include:

- `embed_eval` formalises the property we introduce alongside the embedding operator: that evaluating the result of some embedded delta is the same as evaluating the original expression, and then applying the delta to the value.
- `evalCurve_recover`: at a high level establishes a *conservation of information* property. It, alongside two helper lemmas `optMergeCurve_eval_left` and `optMergeCurve_eval_right`, prove that when the merge operator $\bowtie$ encounters a conflict, moving the residual deltas from the global environment to the AST via embedding does not change the functionality of the program. Formally, this says that if some delta environment Δ can be split into 2 sub-environments Δ_e, Δ_f, then for some curve expression c we have that $\Delta_f \vdash c \odot \Delta_e \Downarrow v'$ is equivalent to $\Delta \vdash c \Downarrow v'$.

PutPut also mainly holds by construction – since the semantics are already based around composing deltas, the proof of this is trivial. Internally, we represent delta compositions as lists, and so end up proving a minor extension of Lean's `List.foldr_append` property.

In conclusion, while the derivation of these proofs is significantly more complicated than the corresponding Sketch-n-Sketch proofs, requiring some structural workarounds, we can conclude that these semantics are undoubtedly more robust. Whereas the SnS semantics require extensive preconditions and frequently violate the laws, the proof of these laws is trivial, hold universally, and only require some supplementary theorems which independently prove useful information.

We do however note that our proof only became practical (even with a state-of-the-art LLM) thanks to the extensive restrictions on our language. As mentioned, we heavily rely on the Normal Form property of our language to be able to prove mere termination of the update semantics, let alone any correctness. This is something that future work should be mindful of: extending either the language or semantics (for example, adding support for curve bindings) may be hindered by this challenge, and require a more thoughtful design or extra restrictions.

4.4 Expressivity

Our DSL is designed for creative coding, and so we evaluate its expressivity by the variety of curves and shapes that can be modelled practically with it.

4.4.1 Discrete Fourier Transformation

We noted in section 2.4.1 that the DFT lets us express any continuous curve as a sum of epicycles, and thus as a term in our language. We demonstrated this at the university Poster Day, creating the majority of our poster *with* the DSL. We used a simple Python script to sample a trace of a font file for text, and the SVG of the University of Bristol logo. We then wrote a simple DFT implementation in Haskell, which converts a list of point traces into a term in our DSL – thanks to the embedded design of our implementation, this is just a normal Haskell program. This, combined with a simple layout system to position curves allowed us to generate an award-winning poster, depicted in fig. 4.1.

We add a DFT-based drawing tool to our UI, which allows users to draw a simple curve which is converted to a term in our DSL via the same method. This improves the creative coding potential of our system, as users can truly iterate in either direction – they can draw a curve, then edit the generated program, or write a program, then edit the generated curve.

However, we should note that the epicycle-based abstraction is not without its flaws. The fundamental problem is that while the DFT can approximate any curve, it becomes very inefficient for certain types of curves, especially any sort of sharp corner. Drawing a square, for example, requires an infinite number of epicycles to be perfectly represented. The Fourier-series representation remains powerful, and expressive in ways one might not initially consider. For instance, we are able to model straight line segments with two epicycles, by giving them inverse frequencies and the same radius. For a simple example, consider $\text{epicycle}(r, \omega, \phi) \oplus \text{epicycle}(r, -\omega, \phi)$. This represents the curve $c(t) = r \cdot e^{2\pi i(\omega t+\phi)} + r \cdot e^{-2\pi i(\omega t+\phi)}$ which simplifies:

$$\begin{aligned} c(t) &= r \cdot e^{2\pi i\omega t+\phi} + r \cdot e^{-2\pi i\omega t+\phi} \\ &= re^{i\phi}(e^{2\pi i\omega t} + e^{-2\pi i\omega t}) \\ &= re^{i\phi}(2\cos(2\pi\omega t)) \\ &= 2r\cos(2\pi\omega t) \cdot e^{i\phi} \end{aligned}$$

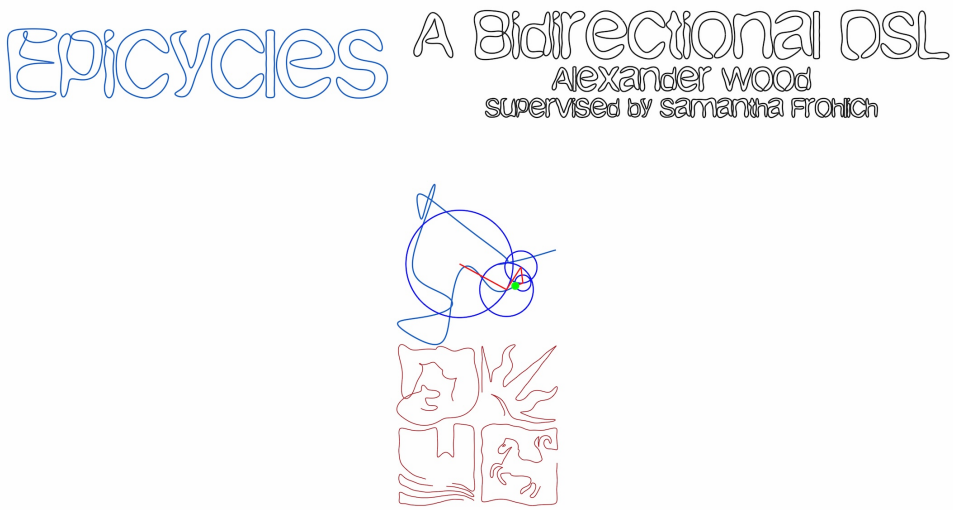


Figure 4.1: A simplified version of our Poster Day poster, generated with the DFT-based functionality of our DSL.

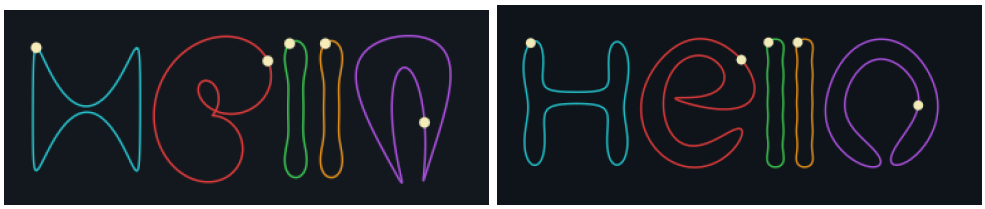


Figure 4.2: The text "Hello" interpolated with the Discrete Fourier Transform with 5 cycles per glyph (left) and 10 cycles per glyph (right).

This essentially is a complex unit vector ($e^{i\phi}$) multiplied by a smoothly oscillating real component $2r\cos(2\pi\omega t)$, and thus traces a straight line at angle ϕ. Formally, this models a specific type of roulette known as a Tusi Couple.

Where the limitation does bite is in encoding efficiency for genuinely polygonal or text-like content: rendering text, for example, needs at least 6 epicycles per character for any kind of recognisable result, and around 10 to be considered a 'good approximation' of real text. We demonstrate this in fig. 4.2.

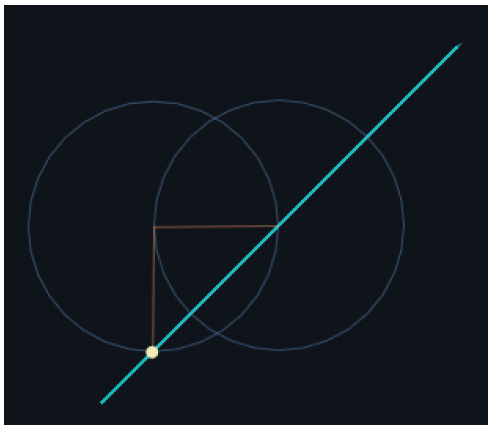
Furthermore, editing the generated programs, while semantically correct (in most cases), does not necessarily translate to an intuitive user experience. Dragging an epicycle in a DFT-generated glyph curve, for example, usually has the effect of distorting the curve to the point of destroying its recognisability as a glyph, rather than producing an interesting transformation of the glyph, which the user might expect. We consider this primarily to be a limitation of the target domain – the ability to approximate text is fragile, and as the primary focus of our DSL is geometric art, largely a byproduct of the Fourier Series representation. We note that future work could include improving the DFT-based functionality to attempt to mitigate this issue, for example by more intelligently utilising the straight line approximations, or adding extra constraints to the generated program to preserve recognisability, however this is outside the scope of this project.

4.4.2 Case Studies

The primary focus with regards to creative coding for our DSL is to be able to express a variety of geometric curves. We evaluate this by presenting a number of case studies, showing different patterns and shapes that can be created with our DSL, alongside their source code.

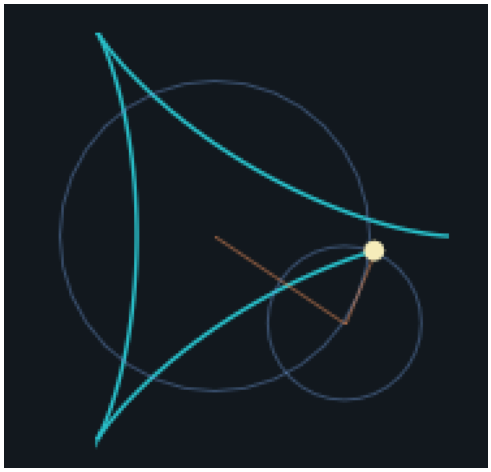
Gallery

We demonstrate a few key examples of the types of images that can be produced with our DSL:



```
straightLine =
  epicycle 80 3 0
    <> epicycle 80 (-3) (pi / 2)
```

Figure 4.3: A straight line at angle $\pi/2$ formed of two epicycles, with construction enabled



```
deltoid =
  epicycle 60 1 0
    <> epicycle 30 (-2) 0
```

Figure 4.4: A deltoid curve formed of two epicycles, with construction enabled

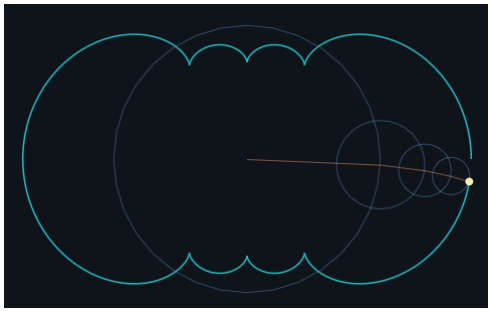
- Figure 4.3 shows the aforementioned straight line from two counter-spinning epicycles.
- Figure 4.4 shows a *Deltoid Curve*. This demonstrates that while sharp corners are difficult to represent, we are still able to approximate some polygonal-like curves
- In fig. 4.5 we introduce the use of shared variables to our gallery, to create an effect of harmonic decay with epicycle radii.
- Figure 4.6 shows a snowflake-like curve generated by exponentially decaying a shared radius parameter across 4 epicycles.
- Figure 4.7 demonstrates that our DSL is not limited to symmetric curves – by sharing a parameter across the phase variable, we create an asymmetric pattern
- Figure 4.8 shows a *Guilloché Curve*[1], a complex spirograph-like curve often used on banknotes.

Parameter Editing

We demonstrate the parameter editing capabilities with a short demonstration of how editing 1 frequency parameter can produce a visually varied response. We use the following demo program and use the GUI editing capabilities to vary the frequency of the second `epicycle`

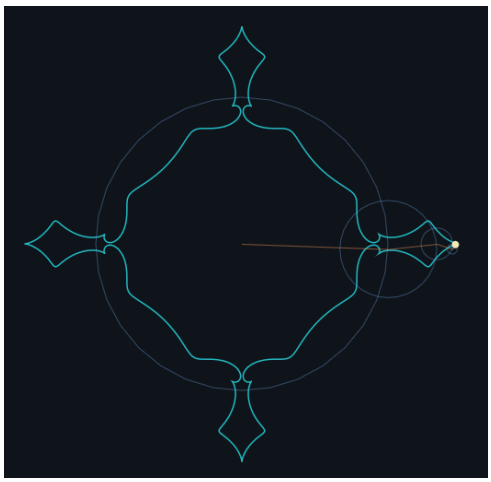
```
demo =
  epicycle 50 1 0
    <> epicycle 50 2 0
```

[1] code loosely adapted from https://mathworld.wolfram.com/GuillochePattern.html



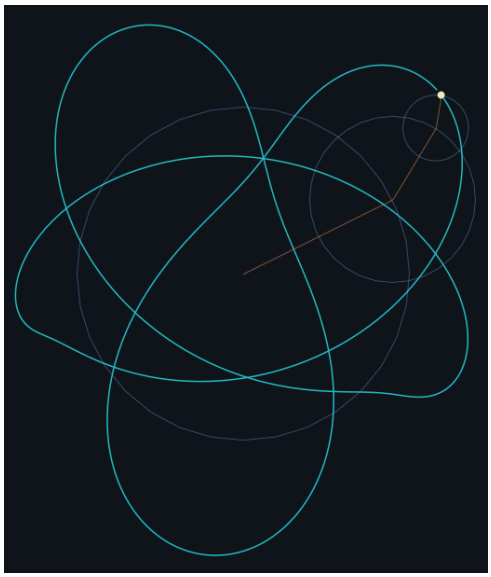
```
curveSharedParameters =
  let_ 60 $ \a ->
    epicycle a 1 0
      <> epicycle (a * recip 3) 3 0
      <> epicycle (a * recip 5) 5 0
      <> epicycle (a * recip 7) 7 0
```

Figure 4.5: A curve with a shared parameter used multiple times. The decreasing radii create a harmonic decay effect.



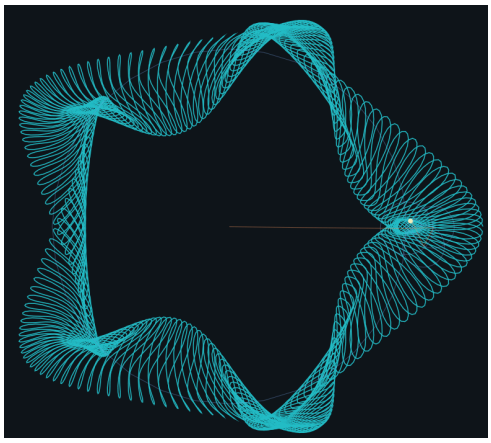
```
snowflake =
   let_ 60 $ \r ->
    epicycle r 1 0
        <> epicycle (r / 3) (-3) 0
        <> epicycle (r / 9) 9 0
        <> epicycle (r / 27) (-27) 0
```

Figure 4.6: A snowflake-like curve generated by exponentially decaying a shared radius parameter across 4 epicycles.



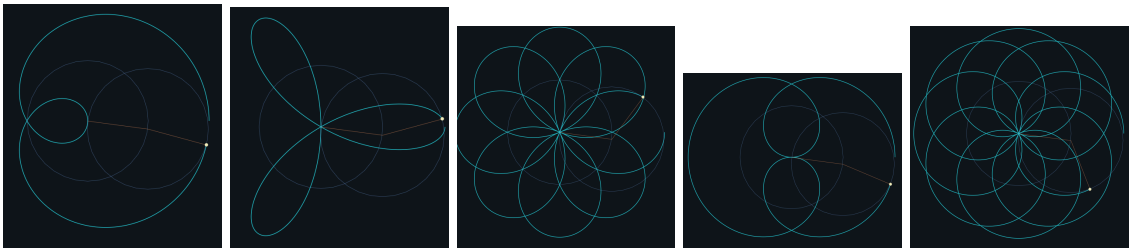
```
asymmetric =
  let_ 0.5 $ \p ->
    epicycle 60 3 p
      <> epicycle 30 (-2) (p * 2)
      <> epicycle 12 7 (p * 3)
```

Figure 4.7: An asymmetric curve generated by sharing a parameter across the phase variable.



```
guilloche =
 let_ 150 $ \r ->
    epicycle r 1 0
        <> epicycle (r * 0.15) (-4) 0
        <> epicycle (r * 0.12) 201 0
        <> epicycle (r * 0.12) (-196) 0
        <> epicycle (r * 0.04) 400 0
```

Figure 4.8: A Guilloché curve generated by sharing a radius parameter across 5 epicycles



(a) $\omega_2 = 2.$ (b) $\omega_2 = -2.$ (c) $\omega_2 = -7.$ (d) $\omega_2 = 3.$ (e) $\omega_2 = 9.$

Figure 4.9: Editing the frequency parameter ω_2 of an epicycle produces a variety of different curves.

Figure 4.9 shows the result of editing the frequency parameter to different values. We can see that a wide variety of curves can be produced by varying a single parameter, showing that our DSL's environment can be used effectively for artistic experimentation.

4.5 Direct Manipulation and User Experience

Some of the biggest pitfalls of our User Experience come as a consequence of our choice to use an Embedded DSL. Chapter 1 discussed live coding systems such as *TidalCycles*, which we took a level of inspiration from, particularly the tight feedback loop that the live coding environment provided. Our system, however is slightly less fluid. The GUI does not provide a text editor for the source code due to the technical complexities of adding one. As our DSL is embedded in Haskell, parsing and evaluating the code relies on the entire GHC compiler and toolchain. Consequently, the curve programs must be within a larger Haskell file, which must be separately compiled and run, and then recompiled and re-ran upon edits.

There is scope for improvement here: *TidalCycles*, for example, decouples the source code and evaluation into two different processes – code can be written in any text editor, and is then 'serialised' to an external daemon which interprets it. We initially considered this architecture, but concluded it would require significant engineering work that would detract from the core focus of the project: the design and formalisation of the DSL and its semantics. Novel solutions such as building a custom Language Server Protocol (LSP) implementation[2] to connect the output and text editor more directly are also worthy of future consideration. However, we acknowledge that the current separation is a limitation of our system.

Furthermore, the deep embedding causes our system to suffer from a lack of lexical round-tripping. Our UI contains a code panel showing the source of the rendered curves, however this is strictly an approximation generated by pretty-printing the AST, and is not the verbatim source code (edited or otherwise). Certain elements of the source code such as binder names cannot be reified at all due to the fundamental limitation of embedding, and so we are forced to assign variables generated names. Additional properties such as white space, comments, etc are also discarded.

[2] https://lambein.xyz/paw-live2023/

These two factors combine to make a full bidirectional round-trip difficult – an 'Export Code' button, for example, would suffer from a number of implementation challenges: the DSL cannot be printed 'as-is', as some extra Haskell preamble is needed – the system would need to be very sensitive to avoid overwriting this. Heavily formatted code would also be overwritten by the system's standardised output, disrupting the programmer's workflow.

This motivates moving to a bespoke DSL with its own parser and editor in future work, which would address these limitations. While we lose the benefits of embedding such as being able to reuse Haskell's type system, the engineering work in reimplementing the subset of features we use would not be prohibitive given more time.

We consider this an acceptable limitation due to our language being relatively minimal, so manually synchronising the updated code to the editor is generally feasible and not overly cumbersome. A simpler 'copy snippet' button would also provide a suitable compromise, particularly for more complex curve definitions such as the result of a hand-drawn trace.

4.6 Metrics

While the formal properties established in Section 4.1 provide theoretical guarantees, we run quantitative metrics on the semantics to measure the practical boundaries and severity of their theoretical limitations. Specifically, we evaluate:

- **Intent Leakage and Reversibility**: How effectively a sequence of edits can be 'reversed', and the magnitude of 'collateral damage' caused to surrounding epicycles
- **AST Growth Bounds**: the increase in the size of the AST after the update, in terms of total node count.
- **PutPut failures**: the rate of PutPut failures, including both asymmetric failures and disagreements, across a number of sequential edits.

Scene Catalogue. We design a small catalogue of test scenes designed to simulate the different strengths and weaknesses of the different semantics. These include a few simple curves as baselines, curves with intense use of shared parameters, and simulating edge cases such as the counterexample discussed in section 4.2.3.

We measure all these metrics over a number of varied *scenarios*, which provide different types of edits. Some scenarios are designed to simulate real user interaction, some are designed for stress testing, and some replicate edge cases that the UI never permits:

- **BigJump**: a large delta between the initial and target value
- **RandomWalk**: a series of random deltas applied sequentially, simulating a user experimenting
- **Drag**: a series of monotonic deltas applied sequentially, simulating a user dragging an epicycle in one direction
- **Oscillate**: a series of alternating positive and negative deltas applied sequentially
- **SiblingChain**: for each adjacent epicycle pair $(i, i+1)$, apply a delta to i then $i+1$ then i then $i+1$, etc.

4.6.1 Quantifying the AST Growth

We formalised in Theorem 2 that the FuseDM-based semantics cause the AST to expand until all shared variables are 'shielded' by synthesised nodes. We investigate whether this expansion renders the resulting source code unreadable or computationally bloated. We find that our theorem holds in practice – the standard deviation of AST growth over 20-25 series of random edits is exactly zero. This proves that over time, the embedding operator does cause the AST to converge to a final fixed point. Furthermore, we observe the size of the AST growth to be manageable: the maximum growth across all scenarios and scenes is approximately 1.4×, going from an initial curve of 24 nodes, to a final of 34 nodes. Some curves exhibited zero growth (if they have no shared variables), and the average growth overall (excluding cases of zero growth) is approximately 1.3×, which we argue is an acceptable and manageable increase that would not impact the readability of the final code.

Scene	Initial	Final Min	Final Max	Final Std	Growth	Runs
AddAlias	8	10	10	0.000	1.250x	20
Ambiguous	33	33	33	0.000	1.000x	25
DeepShared	34	44	44	0.000	1.294x	25
DualRecip	12	12	12	0.000	1.000x	20
LeftBias	19	23	23	0.000	1.211x	25
MulAlias	8	8	8	0.000	1.000x	20
NestedLet	20	28	28	0.000	1.400x	25
Shared	24	34	34	0.000	1.417x	25
Simple	14	14	14	0.000	1.000x	25
SquareAlias	8	10	10	0.000	1.250x	20

Table 4.1: AST Size Growth under FuseDM semantics. The zero standard deviation across runs confirms convergence to a structural fixed point.

4.6.2 Intent Leakage and Reversibility

In contrast, we measure how the semantics handle a 'net-zero' user interaction with the **Oscillate** scenario (as the edits return to the original state). In a perfectly reversible system, this should leave both the AST and the visual output entirely unchanged. We also measure 'intent leakage', which can be viewed as quantifying the magnitude of a violation of the PutGet law when considering the entire curve as the view. In other words, an edit 'leaks' if it causes 'collateral damage' to other epicycles which were not directly interacted with.

Our metrics show that this property comes at odds with semantic purity: the SnS-based strategies (both Heuristic and Naive) demonstrate 100% structural reversibility, returning the AST to its exact initial size and shape. This is to be expected, as the semantics cannot structurally change the AST. However, this comes at the cost of silently corrupting the semantic state – over 50 runs on scenes with shared variables, the Naive update 'leaked' an average of 1080 unintended changes to sibling epicycles. The Heuristic strategy's cost-ranking mitigates this (averaging 330 leaked changes), but does not (and cannot) eliminate it.

Conversely, the FuseDM semantics' AST rewriting causes it to exhibit a mere 40% structural reversibility rate – i.e. in 60% of cases, some edits cause the AST to be permanently altered due to the literal shielding. However, its mean intent leakage is exactly 0.0. This demonstrates that while the SnS approach maintains an illusion of an untouched source file, it more subtly disrupts the (potential) user intention. In contrast, FuseDM sacrifices the original AST shape, potentially adding some user friction, to guarantee that unedited epicycles remain unchanged – potentially better matching the user's intent.

Strategy	Returned %	Mean Leak	Median	Max	Runs
Heuristic	100.0	330.0	0.0	1200	50
Naive	100.0	1080.0	0.0	6000	50
FuseDM	40.0	0.0	0.0	0	50

Table 4.2: Intent leakage and structural reversibility when oscillating on shared scenes.

4.6.3 PutPut Failure

Finally, we evaluate the rate of PutPut failures across a large amount of edits, examining two types of PutPut failure: 'asymmetric failures', when one path succeeds but the other fails (the stability failure mentioned in section 4.2.3), and 'disagreements' when both paths succeed but produce different final states. We observe that once again, the FuseDM-based semantics always satisfy PutPut, whereas the Sketch-n-Sketch-based semantics occasionally fail, at an overall rate of approximately 5.79%. This primarily manifests as end-state disagreements, alongside a smaller 0.61% rate of asymmetric failures where the solver forces itself down invalid mathematical branches. The most significant example of PutPut failure on the SnS-based semantics was with the following program, which failed on almost 70% of edits:

```
curve =
  let_ 10 $ \r ->
    epicycle (r * r) 1 0
```

Strategy	Intermediate OK %	Direct OK %	Asymm %	End Dis %	Combined %
SnS	99.39	100.00	0.61	5.79	5.79
FuseDM	100.00	100.00	0.00	0.00	0.00

Table 4.3: PutPut violation rates across 984 tests.

This exposes the biggest failure of our SnS-based semantics thus far: the locality of the semantics. Just as our initial lens-based prototype failed, this fails so frequently because the semantics are unaware of the structure of the expression. When updating to a new v', the U-Mul rules try to update each r individually to some new r' such that $r' * r' = v'$, blind to the fact that both occurrences refer to the same variable, which produces invalid updates that fail PutGet *and* PutPut in almost all cases. The true solution, $r = \sqrt{v'}$ is never in the candidate set using our rules. This is a fundamental limitation of these semantics, not the heuristics – the valid solution will almost never be in the candidate set. We must either filter out the bad updates, which makes the drag do nothing, or allow them despite being malformed, which causes an unpredictable user experience and 'jitters' in the UI – both solutions are unsatisfactory. Sketch-n-Sketch does not attempt to solve issues like this itself, instead delegating the work to an external Computer Algebra System[8]. We do not do such a thing, and instead accept this as a limitation which motivates the use of the delta-based semantics instead. We note that this issue could be alleviated in many cases by extending the candidate set to attempt some more complex updates – for example, we might introduce some new rule which tries the solution $\sqrt{v'}$ if the expression $e_1 \times e_2$ happens to also evaluate to $e_1 \times e_1$.

$$\frac{\sigma \vdash e_1 \times e_2 : \eta \Downarrow v_1 \qquad \sigma \vdash e_1 \times e_1 : \eta \Downarrow v_2 \qquad v_1 = v_2 \qquad \sigma \vdash e_1 : \eta \Uparrow \sqrt{v'} \rightsquigarrow (\sigma_1, e_1')}{\sigma \vdash e_1 \times e_2 : \eta \Uparrow v' \rightsquigarrow (\sigma_1, e_1' \times e_2)} \text{ U-Mul-Sqrt}$$

This approach does not generalise though – each new rule increases the size of our candidate set exponentially, and just as with our failed prototype, we tend towards reinventing a Computer Algebra System. The possibility of failure is a fundamental limitation of this style of semantics.

4.7 Automated Testing

We also construct a small suite of *property tests*[3] using the *Hedgehog* Haskell library, to further verify our theoretical claims. For example, we write some tests about exactly how many candidates the SnS update returns for a given AST and edit. The majority of these tests are properties we have already formally proven, but these tests serve as an extra sanity check, catching engineering bugs in our implementation

4.8 Summary

We conclude that for our particular domain, and bidirectional programming as a whole, the delta-based semantics are superior to the Sketch-n-Sketch ones, due to its more lawful behaviour, which makes it easier to reason about, and more intuitive for the user. We consider the many extra cases required to prove basic properties like GetPut and PutGet on the SnS semantics, due to the numerous edge cases, which, in some situations, manifest as an unacceptably high failure rate of editing.

Chapter 5

Conclusion

Our core thesis is that a well-chosen domain restriction makes the View-Update problem tractable. We demonstrate this by designing and implementing a DSL and UI for bidirectional creative coding geometric art by composing epicycles. We show that our DSL is expressive enough to model a variety of visually interesting curves, and that our UI allows intuitive and unambiguous direct manipulation of these curves.

On a technical level, we use modern, advanced techniques to implement our embedded DSL in Haskell, with a high-level HOAS interface which 'compiles' to a lower level FOAS AST, allowing for a readable frontend language without losing introspective power. We use industry-standard design patterns such as *Trees that Grow* to encode the invariants of our DSL into Haskell's type system, making invalid states unrepresentable – our implementation contains zero partial functions or unsafe pattern matches (excluding tests).

We implement two different semantics for bidirectional evaluation of our DSL, one based on the non-deterministic Sketch-n-Sketch system, and another based on the AST-rewriting FuseDM system, satisfying our first two aims in our introduction.

We adjust these semantics to work with our language, and prove common correctness properties such as PutGet and GetPut on them with the Lean theorem prover. We also evaluate the two semantics quantitatively, running them on a large test suite and evaluating their success and failure rates in the context of creative coding. We discuss the flaws of each system in depth both theoretically and for practical usage, satisfying our third aim.

Finally, we evaluate the expressivity of our DSL, exploring the creative potential of our domain restriction. We demonstrate that we can model a myriad of curves, and can use the Discrete Fourier Transform to approximate things like text, images, and user-drawn curves. We identify the weaknesses of the domain, such as inefficient or unintuitive encodings of more complex curves, and visual fragility when editing due to a lack of constraints.

A key outcome of this evaluation is the identification of a fundamental trade-off in bidirectional systems: the choice between *structural reversibility* and *intent preservation*. While the Sketch-n-Sketch approach maintains the original source code's shape, it produces updates which can 'corrupt' other parts of the program, causing the original intent to degrade. In contrast, the delta-based approach prioritises matching the user's intent as closely as possible, at the cost of making structural changes to the program. We argue that in a creative context, where the visual output is the primary object of concern, the penalty of this 'literal shielding' is a small price to pay for the more intuitive semantics.

The future work identified in this project is clear. The simple interaction model of our system does not allow the power of the delta-based semantics to fully show themselves: although we support a more complex family of deltas, there is no way to trigger these from the UI. This was mainly due to designing the interaction model around the simpler Sketch-n-Sketch based semantics, which may struggle with more complex edits. Moving forward, we would focus on the delta-based semantics and exploring how to expand the system and use it more effectively. For example, we would investigate addressing the fragility of the text approximation by exploring ways to add constraints to updates. This could tie in with our observation that we under-utilise the delta semantics; for example, we could explore ways of grouping updates, where an edit to one part of a curve propagates a synchronised delta to a cluster of related epicycles, preserving the structure while still allowing updates. We also noted that while the PutGet law is valuable from a theory perspective, sometimes the user does *want* to violate it when they have defined a shared variable – a useful future direction would be to explore ways of allowing the user to explicitly choose when to violate PutGet in a controlled way that still behaves predictably. One option would be

taking the opposite approach to Sketch-n-Sketch: rather than having a ! operator to 'lock' values, we have an operator to 'unlock' them, allowing them to be edited in ways that violate PutGet, but only when the user explicitly opts in to this behaviour. Any additions in this direction should carefully evaluate the amount of work they force the user to do – our system succeeds in part because it 'just works', and adding more and more extra tools to the kit could cause more friction when creative coding.

A promising future avenue would be to take the lessons we have learned from this project to a new, less restricted creative domain. For example, we may revisit our failed initial prototype with a delta based approach to investigate how far we can take it with a more thoughtful design. Informal user feedback of Epicycles also identified some more practical examples of where a system like this may be useful, namely drawing TikZ diagrams in LaTeX– a visual domain with a reputation for poor feedback loops and no support for direct manipulation.

In conclusion, this project demonstrates that by carefully restricting the programming and artistic domains, we can bridge the 'abstraction gap' between code and visual output in such a way that allows for tractable solutions to the View-Update problem, without sacrificing expressivity. While general-purpose bidirectional programming remains an open challenge, the success of our DSL suggests that domain-specific solutions provide a practical, lawful, and expressive path for the next generation of creative tools.

Appendix A

Appendix A: AI Prompts/Tools

The font and SVG sampling scripts discussed in section 4.4.1 were written with the assistance of Google Gemini. The model was given a function signature of the Haskell DFT functions we wrote, and told to make scripts that sample a trace of a font and an SVG, and emit a valid Haskell source file which contains definitions of the sampled curves. We then wrote a very thin wrapper which imports the generated source file and runs the DFT to generate terms in our DSL.

We also turned to LLMs for the mechanisation of the proofs of the delta semantics, which we have discussed in section 4.3. These proofs were guided by us, including providing information about the structure of the proof that made the model produce a usable output – without this extra information, it came up with incorrect or unsatisfactory results. We manually verified the output, checking for any red-flags, and made an effort to understand it at a high level, along with explaining it. Furthermore, we emphasise that the proofs for the Sketch-n-Sketch semantics were done without the assistance of AI and are kept separate in the code. To avoid any confusion, all Lean files in the supplementary material are manually marked with a comment noting if they were written by AI or not.

Bibliography

[1] Jacques Carette, Oleg Kiselyov, and Chung-Chieh Shan. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *Journal of Functional Programming*, 19(5):509–543, September 2009. ISSN 0956-7968, 1469-7653. doi: 10.1017/S0956796809007205. URL https://www.cambridge.org/core/product/identifier/S0956796809007205/type/journal_article.

[2] Ravi Chugh, Brian Hempel, Mitchell Spradlin, and Jacob Albers. Programmatic and Direct Manipulation, Together at Last. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 341–354, June 2016. doi: 10.1145/2908080.2908103. URL http://arxiv.org/abs/1507.02988. arXiv:1507.02988 [cs].

[3] Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*, pages 268–279. ACM, September 2000. ISBN 978-1-58113-202-1. doi: 10.1145/351240.351266. URL https://dl.acm.org/doi/10.1145/351240.351266.

[4] N.G De Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972. ISSN 13857258. doi: 10.1016/1385-7258(72)90034-0. URL https://linkinghub.elsevier.com/retrieve/pii/1385725872900340.

[5] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Trans. Program. Lang. Syst.*, 29(3):17–es, May 2007. ISSN 0164-0925. doi: 10.1145/1232420.1232424. URL https://dl.acm.org/doi/10.1145/1232420.1232424.

[6] Enrico Franconi and Paolo Guagliardo. The View Update Problem Revisited, November 2012. URL http://arxiv.org/abs/1211.3016. arXiv:1211.3016 [cs].

[7] Mark Guzdial. A media computation course for non-majors. In *Proceedings of the 8th annual conference on Innovation and technology in computer science education*, pages 104–108, Thessaloniki Greece, June 2003. ACM. ISBN 978-1-58113-672-2. doi: 10.1145/961511.961542. URL https://dl.acm.org/doi/10.1145/961511.961542.

[8] Brian Hempel, Justin Lubin, and Ravi Chugh. Sketch-n-Sketch: Output-Directed Programming for SVG. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology*, pages 281–292, New Orleans LA USA, October 2019. ACM. ISBN 978-1-4503-6816-2. doi: 10.1145/3332165.3347925. URL https://dl.acm.org/doi/10.1145/3332165.3347925.

[9] Martin Hofmann, Benjamin Pierce, and Daniel Wagner. Edit lenses. *SIGPLAN Not.*, 47(1):495–508, January 2012. ISSN 0362-1340. doi: 10.1145/2103621.2103715. URL https://dl.acm.org/doi/10.1145/2103621.2103715.

[10] Furio Honsell, Marino Miculan, and Ivan Scagnetto. The Theory of Contexts for First Order and Higher Order Abstract Syntax. *Electronic Notes in Theoretical Computer Science*, 62:116–135, June 2002. ISSN 15710661. doi: 10.1016/S1571-0661(04)00323-8. URL https://linkinghub.elsevier.com/retrieve/pii/S1571066104003238.

[11] Kazutaka Matsuda, Samantha Frohlich, Meng Wang, and Nicolas Wu. Embedding by Unembedding. *Proceedings of the ACM on Programming Languages*, 7(ICFP):1–47, August 2023. ISSN 2475-1421. doi: 10.1145/3607830. URL https://dl.acm.org/doi/10.1145/3607830.

[12] Mikaël Mayer, Viktor Kuncak, and Ravi Chugh. Bidirectional evaluation with direct manipulation. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–28, October 2018. ISSN 2475-1421. doi: 10.1145/3276497. URL https://dl.acm.org/doi/10.1145/3276497.

[13] Alex McLean and Geraint Wiggins. Tidal–pattern language for the live coding of music. In *Proceedings of the 7th sound and music computing conference*, pages 331–334, 2010.

[14] Shayan Najd and Simon Peyton Jones. Trees That Grow, October 2016. URL http://arxiv.org/abs/1610.04799. arXiv:1610.04799 [cs].

[15] F. Pfenning and C. Elliott. Higher-order abstract syntax. *ACM SIGPLAN Notices*, 23(7):199–208, July 1988. ISSN 0362-1340, 1558-1160. doi: 10.1145/960116.54010. URL https://dl.acm.org/doi/10.1145/960116.54010.

[16] Juan Ponce Campuzano. Tracing closed curves with epicycles: A fun application of the Discrete Fourier Transform. 11:1–16, January 2023.

[17] Casey Reas and Ben Fry. Processing: programming for the media arts. *AI & SOCIETY*, 20(4): 526–538, September 2006. ISSN 0951-5666, 1435-5655. doi: 10.1007/s00146-006-0050-9. URL http://link.springer.com/10.1007/s00146-006-0050-9.

[18] Shneiderman. Direct Manipulation: A Step Beyond Programming Languages. *Computer*, 16(8): 57–69, August 1983. ISSN 0018-9162. doi: 10.1109/MC.1983.1654471. URL http://ieeexplore.ieee.org/document/1654471/.

[19] Xing Zhang, Ruifeng Xie, Guanchen Guo, Xiao He, Tao Zan, and Zhenjiang Hu. Fusing Direct Manipulations into Functional Programs. *Proceedings of the ACM on Programming Languages*, 8 (POPL):1211–1238, January 2024. ISSN 2475-1421. doi: 10.1145/3632883. URL https://dl.acm.org/doi/10.1145/3632883.